

บทที่ 2 ทฤษฎีที่เกี่ยวข้อง

2.1 บทนำ

ในปัจจุบันความก้าวหน้าทางด้านการประมวลผลสัญญาณต่าง ๆ ได้รุดหน้าไปอย่างรวดเร็ว ซึ่งในบางโอกาสสัญญาณอาจถูกประมวลผลในระบบอนาล็อก (Analog) ในบางโอกาสก็ถูกประมวลผลในระบบดิจิทัลด้วยเครื่องคอมพิวเตอร์ ซึ่งในระบบของการประมวลผลด้วยเครื่องคอมพิวเตอร์ สัญญาณที่ถูกประมวลผลจะถูกแปลงจากสัญญาณอนาล็อกที่ต่อเนื่องไปเป็นสัญญาณดิสครีต (Discrete) มีการสุ่มเอาตัวอย่างสัญญาณไปประมวลผล ซึ่งอยู่ในส่วนการทำงานของการ์ดเสียง จากนั้นจะเป็นขั้นตอนการประมวลผลซึ่งมีมากมายหลายวิธี ซึ่งล้วนแต่มีวัตถุประสงค์ในการประมวลผลสัญญาณเพื่อแยกแยกสัญญาณต่าง ๆ ไปทำประโยชน์ตามต้องการ

2.2 ศาสตร์ด้านการรู้จำ

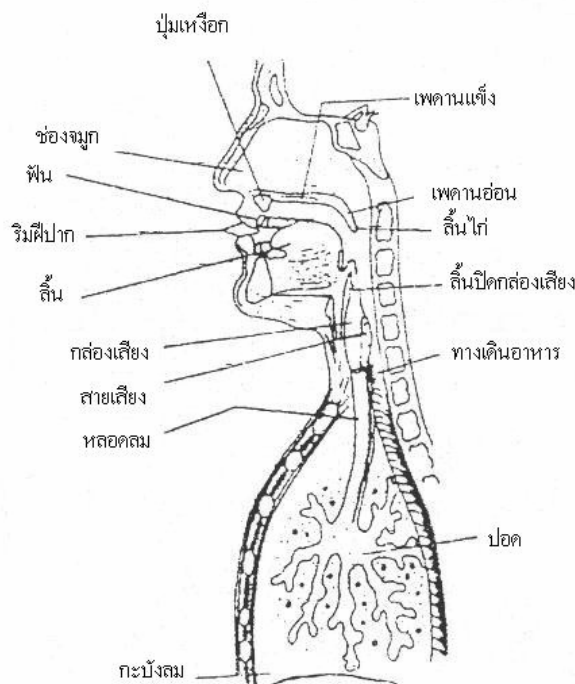
ศาสตร์ด้านการรู้จำ หมายถึง การให้คอมพิวเตอร์รู้จักรูปแบบของสิ่งใดสิ่งหนึ่ง โดยเก็บบันทึกคุณลักษณะบางอย่างเพื่อให้สามารถบอกได้ว่าสิ่งนั้นๆ คืออะไร ซึ่งที่จริงแล้วมนุษย์เองไม่ได้ทราบถึงคุณลักษณะพิเศษเหล่านั้น ในการแยกแยะตามธรรมชาติ ศาสตร์ด้านการรู้จำในช่วงแรกๆ นั้นจะนำหลักสถิติมาใช้ แต่พบ มีขีดจำกัดค่อนข้างมากในการเลียนแบบการรู้จำของมนุษย์ เพราะสมอง ปัญญา และความฉลาดของมนุษย์ ไม่ใช่เรื่องของสถิติเพียงอย่างเดียว

2.3 การแปลงเสียงของมนุษย์

จากการศึกษาด้านกายวิภาคศาสตร์ของมนุษย์ (human anatomy) วิชาที่ว่าด้วยเสียงของภาษา (phonetics) และศาสตร์ทางด้านเสียง (acoustics) ช่วยให้เข้าใจขั้นตอนการทำงานร่วมกันของอวัยวะต่างๆ

ในการแปลงเสียงพูด ตลอดจนคุณลักษณะทางกายภาพของเสียงพูดเพื่อนำมาวิเคราะห์ และสร้างแบบจำลองเลียนแบบเสียงพูดของมนุษย์

การทำให้เกิดเสียงเป็นหน้าที่หนึ่งของระบบหายใจ การออกเสียงหรือการพูดของมนุษย์แต่ละครั้ง จะต้องมีการทำงานร่วมกันของอวัยวะต่างๆ ของร่างกาย ดังรูปที่ 2.1 อันประกอบด้วย

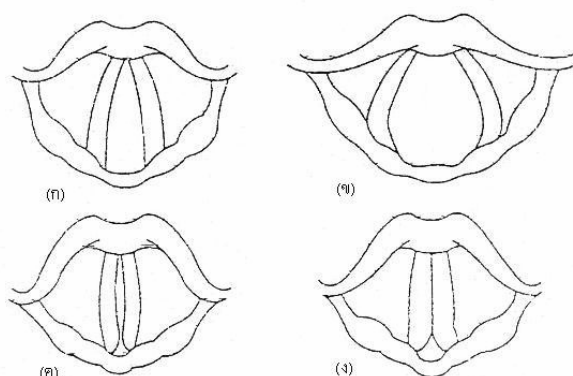


รูปที่ 2.1 ภาพตัดขวางแสดงอวัยวะในระบบการพูดของมนุษย์

1. ปอดและกระบังลม ทำหน้าที่สำคัญในการหายใจ และเป็นต้นกำเนิดการไหลของอากาศในกระบวนการผลิตเสียง

2. หลอดลม ทำหน้าที่นำอากาศจากปอดผ่านกล่องเสียง และเป็นอวัยวะที่อยู่ด้านหน้าของหลอดอาหาร

3. กล่องเสียง เป็นอวัยวะพิเศษที่ทำหน้าที่เป็นทางเดินอากาศเวลาหายใจ และเป็นตัวผลิตพัลส์ (pulse) ของอากาศขณะเปล่งเสียง ซึ่งประกอบด้วยเส้นเสียง (vocal cords) และช่องสายเสียง (glottis) รูปร่างของกล่องเสียง และเส้นเสียงในลักษณะต่าง ๆ แสดงในรูปที่ 2.2



รูปที่ 2.2 รูปกล่องเสียงขณะ

(ก) หายใจปกติ (ข) หายใจเข้าลึก ๆ

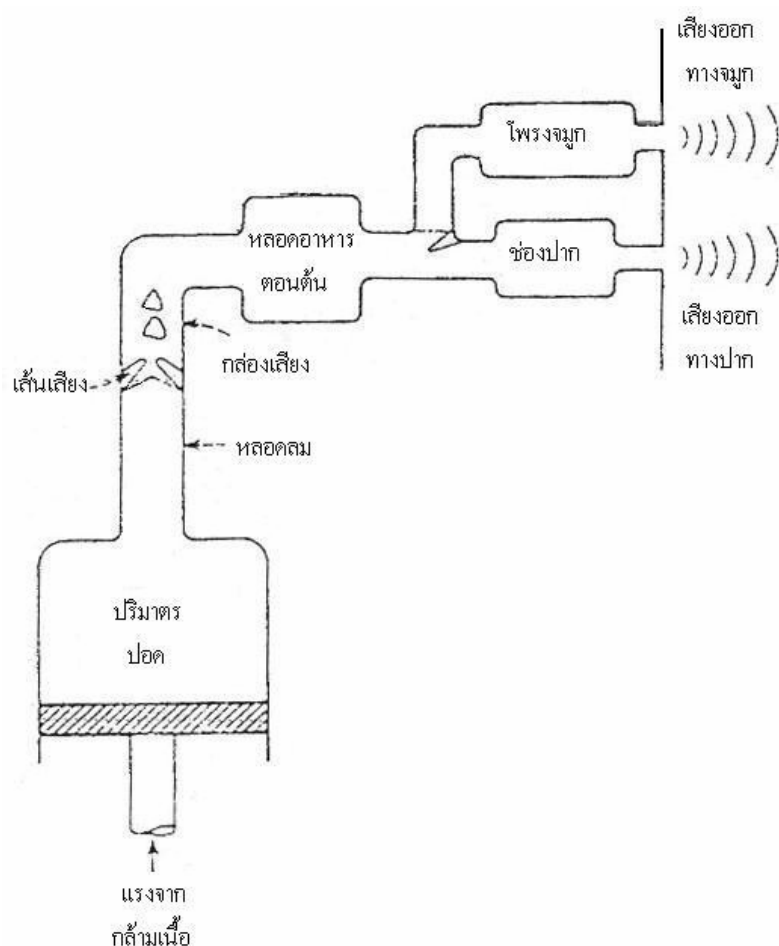
(ค) กำลังส่งเสียง (ง) ส่งเสียงกระซิบหรือเสียงแผ่ว

4. ช่องปากและส่วนของหลอดอาหารตอนต้น อวัยวะกลุ่มนี้อยู่ต่อจากกล่องเสียง อาจเรียกว่าอวัยวะกำทอนเสียง (vocal tract) ทำหน้าที่กำทอนเสียง โดยให้กำทอนทั้งเสียงที่เกิดจากกล่องเสียง และเสียงที่เกิดภายในช่องปาก ขนาดของอวัยวะกำทอนเสียงขึ้นอยู่กับตำแหน่งของลิ้น ริมฝีปาก ขากรรไกร และเพดานอ่อน และเปลี่ยนแปลงไปตามการออกเสียง

5. โพรงจมูก เริ่มจากเพดานอ่อนจนถึงรูจมูกทั้งสอง ทำหน้าที่กำทอนเสียงร่วมกับช่องปาก เมื่อมีการเปล่งเสียงที่ออกจากจมูก (nasal sounds) เช่นเสียง /ม/, /น/ และ /ง/ เป็นต้น

2.4 กระบวนการผลิตเสียงพูด

จากระบบเสียงพูด สามารถแสดงเป็นแผนภาพของระบบกำเนิดเสียง ดังรูปที่ 2.3



รูปที่ 2.3 แผนภาพระบบเสียงพูดของมนุษย์

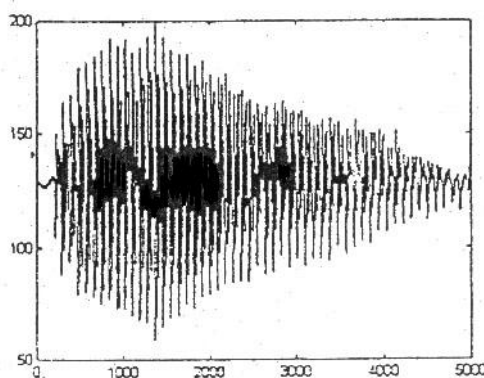
ซึ่งสามารถจำแนกกลไกการสร้างเสียงพูดของมนุษย์ได้ 3 แบบ ดังนี้

1. อากาศที่ไหลจากปอดจะถูกมอดูเลต (modulate) โดยการสั่งของเส้นเสียงทำให้เกิดคลื่นเสียงลักษณะคล้ายพัลส์ที่มีคาบเวลาแบบควาไซ (quasi-periodic pulse-like excitation)
2. อากาศที่ไหลจากปอดถูกทำให้ปั่นป่วนด้วยการบังคับให้ไหลผ่านช่องแคบอันเกิดจากการบีบตัวของอวัยวะในช่องปากทำให้เกิดเสียงลักษณะคล้ายเสียงรบกวน (noise-like excitation)
3. อากาศที่ไหลถูกกัก และเกิดแรงดันอยู่ภายในส่วนของช่องปากที่ปิด จากนั้นจึงปล่อยให้อากาศที่มีแรงดันพุ่งออกไปอย่างรวดเร็ว ทำให้เกิดการกระตุ้นเป็นเสียงในช่วงเริ่มต้น (transient excitation)

2.5 เสียงพูดของมนุษย์

เสียงพูดเป็นคลื่นตามยาว (longitudinal wave) เกิดจากการสั่นของอนุภาคตัวกลางนั้นคืออากาศ และทิศทางการสั่นของอนุภาค จะอยู่ในทิศเดียวกันกับทิศทางของการเคลื่อนที่ คลื่นเสียงเป็นคลื่นที่เปลี่ยนแปลงไปตามเวลา เสียงพูดแบ่งออกได้เป็น 2 ชนิด ตามการกำเนิดเสียง หรือโหมด (mode) การกระตุ้น คือ

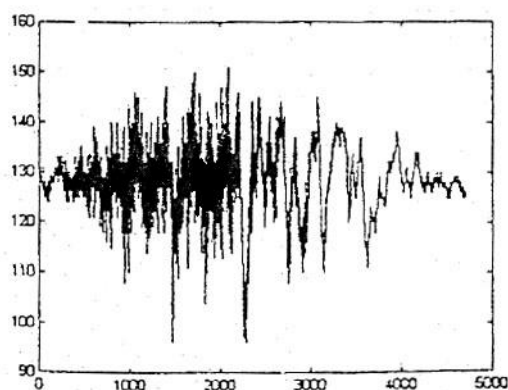
1. เสียงก้องหรือเสียงโหมะ (voiced) เกิดจากการบังคับอากาศให้ผ่านช่องสายเสียงทำให้เกิดการเปลี่ยนแปลงความตึงหย่อนของเส้นเสียง โดยเส้นเสียงจะสั่นและเกิดเป็นพัลส์ (pulse) ของอากาศไปกระตุ้นอวัยวะกำหนดเกิดเป็นเสียงก้อง ตัวอย่างเสียงก้องได้แก่ เสียงสระ เสียงพยัญชนะ ที่ต้องออกเสียงจากลำคอ (voiced consonants) เช่น เสียง /a/ ดังรูป 2.4 (ก)



รูปที่ 2.4 (ก) ตัวอย่างรูปคลื่นของสัญญาณเสียงก้อง /a/

2. เสียงไม่ก้องหรือโหมะ (unvoiced หรือ voiceless) เป็นเสียงที่เกิดในช่องปาก หรือโพรงจมูก โดยอวัยวะภายในช่องปาก ริมฝีปาก ขวาการไหลของอากาศให้ผ่านได้เป็นช่องเล็ก ๆ อากาศจึงไหลผ่านอย่างรวดเร็ว และปั่นป่วนจนกระทั่งสร้างเป็นเสียงรบกวนช่วงความถี่กว้าง (broad-

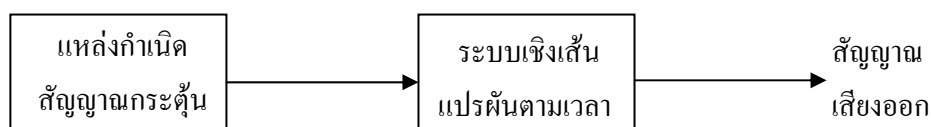
spectrum noise) ตัวอย่างเสียงไม่ก้องได้แก่ เสียงพยัญชนะที่ไม่ได้เกิดจากลำคอ (voiceless consonants) เช่น เสียง /sh/. ดังรูปที่ 2.4 (ข)



รูปที่ 2.4 (ข) ตัวอย่างรูปคลื่นของสัญญาณเสียงไม่ก้อง /sh/

2.6 แบบจำลองระบบกำเนิดเสียงพูด

จากภาพรูปที่ 2.5 สามารถแสดงภาพกรอบจำลองระบบกำเนิดเสียงเบื้องต้น โดยมีการแยกภาคแหล่งกำเนิดสัญญาณกระตุ้น ออกจากส่วนกำหนดเสียง ซึ่งแทนด้วยระบบเชิงเส้นแปรผันตามเวลา



รูปที่ 2.5 แผนภาพกรอบจำลองระบบกำเนิดเสียงเริ่มต้น

จากรูปที่ 2.5 แหล่งกำเนิดสัญญาณกระตุ้นทำหน้าที่แทนการทำงานของปอด และกล่องเสียงส่วนนี้จะผลิตขบวนพัลส์ที่มีคาบเวลาพิชัษณะเปล่งเสียง และให้กำเนิดเสียงซึ่งคล้ายเสียงรบกวนขณะเปล่งเสียงอโหะ (unvoiced) ส่วนที่สองเป็นท่อกำหนดเสียง จะแทนการทำงานของช่องปาก และโพรงจมูก ทำหน้าที่เสมือนตัวกรองสัญญาณ (filter) ที่ยอมให้ความถี่ฟอร์แมนต์ผ่านได้ ซึ่งสามารถแทนด้วยระบบเชิงเส้นแปรผันตามเวลา (time-varying linear system)

2.7 รูปแบบของไฟล์เสียง

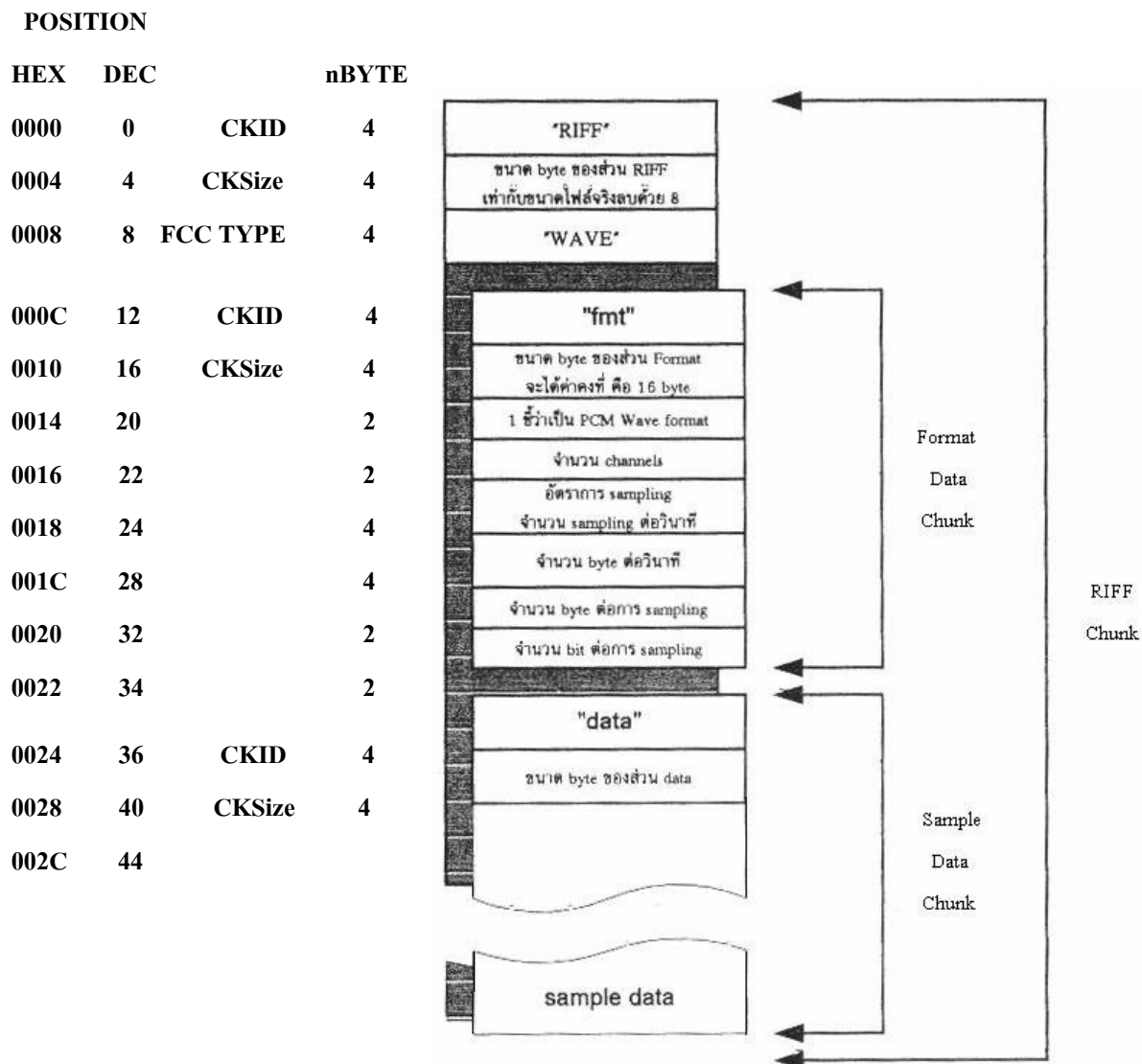
ในการบันทึกเสียงพูดเป็นข้อมูลดิจิทัลโดยใช้การ์ดเสียง จะทำให้ได้เวฟไฟล์ที่มีรูปแบบของไฟล์ RIFF (Resource International File Format) ซึ่งเป็นรูปแบบมาตรฐานที่ใช้กันอย่างกว้างขวาง ไฟล์ RIFF นั้นมีโครงสร้างเชิงซ้อนคือ มีลักษณะเป็นกลุ่ม (Chunk) ซึ่งภายในกลุ่มนี้ยังประกอบด้วยกลุ่มย่อยซึ่งมีโครงสร้างคล้ายกัน ดังแสดงในรูปที่ 2.6

จากรูปที่ 2.6 จะเห็นว่า กลุ่ม RIFF (RIFF Chunk) จะเป็นกลุ่มใหญ่ที่สุด ซึ่งประกอบด้วยกลุ่มรูปแบบของข้อมูล (Format Data Chunk) และกลุ่มข้อมูลจากการแซมปลิง (Sample Data Chunk) ซึ่งในแต่ละกลุ่มจะมีโครงสร้างที่ประกอบด้วย

1. ชื่อกลุ่ม (CKID : Chunk Identification) เช่น RIFF , fmt_ , data ซึ่งมีกฎกำหนดว่าต้องมี 4 ตัวอักษร (FCC : Four Character Code) ถ้ามี 3 ตัวอักษรก็ต้องเพิ่มช่องว่าง (Blank) เข้าไปอีก 1 ตัวอักษรให้รวมเป็น 4 ตัวอักษรในกรณี fmt_

2. ตัวเลขบอกขนาดข้อมูลของกลุ่ม (CKSize : Chunk Size) มีหน่วยเป็นไบต์ (Byte) โดยจะไม่รวมไบต์ที่ใช้ไปในการเก็บชื่อกลุ่ม (CKID) แต่จะเก็บตัวเลขบอกขนาดของกลุ่มเองด้วย ตัวเลขบอกขนาดของกลุ่มจะแสดงด้วยข้อมูล 4 ไบต์โดยไบต์ที่มีความสำคัญต่ำสุด (Least Significant Byte) จะถูกเก็บเข้าไปก่อน

3. ข้อมูลของกลุ่ม (CKData : Chunk Data) มีจำนวนไบต์เท่ากับตัวเลขที่แสดงในข้อที่ 2 เช่น ในกลุ่มรูปแบบของข้อมูล (Format Data Chunk) จะมีข้อมูลของกลุ่มเป็นรูปแบบในการบันทึกเสียง ยกตัวอย่างเช่น อัตราแซมปลิง , จำนวนบิตต่อ 1 แซมเปิล เป็นต้น หรือในกลุ่มข้อมูลจากการแซมปลิง (Sample Data Chunk) จะมีข้อมูลของกลุ่มเป็นแอมพลิจูด (Amplitude) ที่ได้จากการแซมปลิง



รูปที่ 2.6 แสดงโครงสร้างเวฟไฟล์ที่มีรูปแบบของไฟล์ RIFF

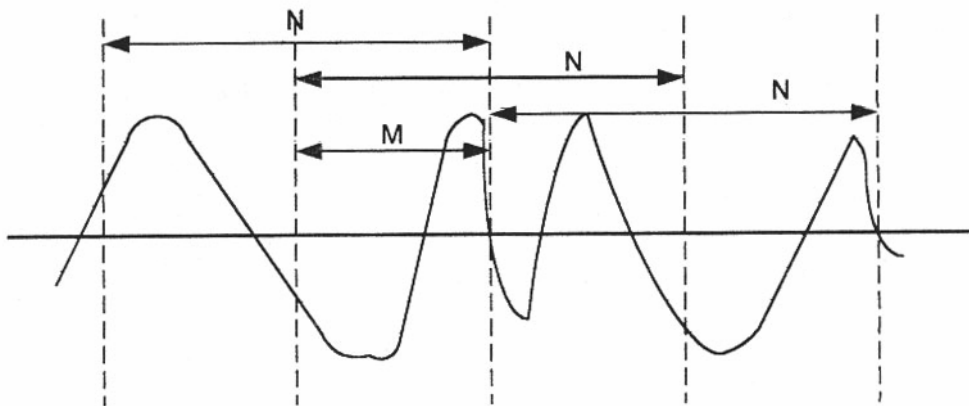
รูปแบบการบันทึก	ค่าสูงสุด	ค่าต่ำสุด	ค่าที่ตำแหน่งตรงกลาง
8 บิต พืชีเอ็ม	255 (FFH)	0	128 (80H)
16 บิต พืชีเอ็ม	32767 (7FFFH)	-32768 (-800H)	0

รูปที่ 2.7 แสดงค่าสูงสุด ค่าต่ำสุด ค่ากลางของรูปแบบการบันทึกแต่ละอัน

2.8 การแบ่งช่วงสัญญาณ (Frame Blocking)

ข้อมูลเสียงซึ่งอยู่ในโดเมนเวลา จะถูกตัดมาวิเคราะห์ทีละเฟรม เฟรมละ N จุด โดยมีจุดเหลื่อมกัน M จุด จนหมดสัญญาณเสียงที่นำมาวิเคราะห์ การเลือกใช้ค่า N และ M พิจารณาได้ดังนี้

- ก) ค่า N เล็กไป จะทำให้ได้สเปกตรัมที่ไม่ละเอียด (Low Resolution) แต่ถ้าค่า N ใหญ่ไป ก็จะลดความสามารถในการติดตามการเปลี่ยนแปลงของสเปกตรัมลง จึงไม่เหมาะสมกับสัญญาณที่มีสเปกตรัมเปลี่ยนแปลงรวดเร็ว
- ข) M เล็กไป จะลดความสามารถในการติดตามการเปลี่ยนแปลงของสเปกตรัมลง แต่ถ้า M ใหญ่ไป ก็จะสิ้นเปลืองการประมวลผลมาก



รูปที่ 2.8 แสดงการแบ่งช่วงสัญญาณ

2.9 การวินโดว์ (Windowing)

เนื่องจากเราใช้วิธีตัดสัญญาณเพื่อมาหาสเปกตรัมเป็นบล็อก ๆ ซึ่งเท่ากับเป็นการหาสเปกตรัมของ “บล็อกของสัญญาณ” ไม่ใช่ตัวสัญญาณจริง ๆ ที่เข้ามาติดต่อกัน ไม่มีจุดเริ่มต้นหรือสิ้นสุด การตัดสัญญาณเป็นบล็อกนี้จะทำให้เกิดความคลาดเคลื่อนในสเปกตรัมที่ได้ วิธีลดผลของความคลาดเคลื่อนนี้ทำได้โดยคูณสัญญาณแต่ละบล็อก หรือ $X_i(n)$ ด้วยฟังก์ชันหน้าต่าง (window function) ก่อนที่จะทำการหาสเปกตรัม

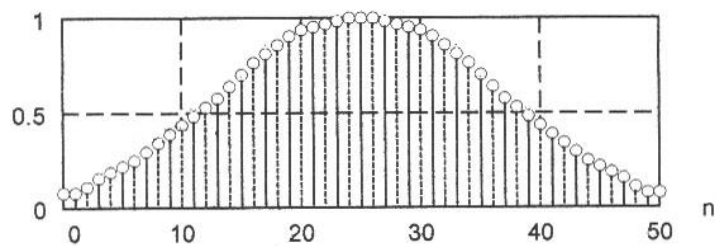
ฟังก์ชันหน้าต่างมีหลายแบบ เป็นฟังก์ชันที่มีลักษณะสมมาตร และมีค่าสูงสุดที่จุดกึ่งกลาง มีความยาวเท่าไรก็ได้ตามต้องการ (ตามค่าที่แทนลงในสูตร) ตัวอย่างของฟังก์ชันหน้าต่างที่เป็นที่นิยมก็คือ หน้าต่างแฮมมิง (Hamming window) ซึ่งมีสมการคือ

$$w(n) = 0.54 - 0.46 \cos\left(\frac{2\pi n}{N-1}\right) \quad ; n = 0, 1, 2, \dots, N-1 \quad (2.1)$$

$$x_{inew}(n) = x_i(n)w(n) \quad (2.2)$$

เราต้องการ $w(n)$ ที่มีความยาวเท่ากับ 1 บล็อกสัญญาณ ดังนั้นต้องใช้ N เท่ากับความยาว 1 บล็อกสัญญาณ รูปร่างของหน้าต่างแบบแฮมมิงแสดงดังรูปที่ 2.9 ซึ่งจะสังเกตเห็นได้ว่า ฟังก์ชันหน้าต่าง

มีค่าเล็กที่ส่วนต้นและส่วนปลาย ซึ่งเป็นส่วนที่ลดผลของการเปลี่ยนแปลงที่จุดเริ่มต้น และจุดสิ้นสุดของบล็อก



รูปที่ 2.9 ตัวอย่างของฟังก์ชันหน้าต่างแบบแฮมมิง ที่ $N = 50$

2.10 การวิเคราะห์ฟูริเยร์ (Fourier Analysis)

การวิเคราะห์ฟูริเยร์ช่วงสั้นเป็นเทคนิคการหาความถี่ที่ใช้กันมานานแล้ว การวิเคราะห์ฟูริเยร์ให้ตัวแทนของสัญญาณเสียงพูดเป็นฟังก์ชันความถี่ในเทอมของขนาดและเฟส เนื่องจากเสียงพูดไม่ได้นิ่งตลอดเวลา ดังนั้นจึงจำเป็นต้องทำการวิเคราะห์ในช่วงสั้น ๆ โดยใช้วินโดว์

การแปลงฟูริเยร์ช่วงสั้นมีสมการดังนี้

$$X_n(e^{-j\omega}) = \sum_{m=-\infty}^{\infty} x(m)[\exp(-j\omega m)w(n-m)] \quad (2.3)$$

ในการคำนวณต้องใช้ DFT แทนการแปลงฟูริเยร์แบบต่อเนื่อง โดยใช้ฟังก์ชันวินโดว์ลดข้อมูลแบบไม่ต่อเนื่องทั้งหมดให้เหลือจำนวน N ตัว (N คือช่วงเวลาหรือขนาดของวินโดว์ที่ใช้ในการแปลง DFT) ข่าวดาต่าง ๆ ใน $X_n(e^{-j\omega})$ จะไม่สูญหายไปจากข้อมูลเดิมถ้าการแปลงนั้นสุ่มมาด้วยความถี่สูงเพียงพอ (คือช่วงระยะห่างระหว่าง N) และวินโดว์ $w(n)$ ไม่มีจุดสุ่มที่เป็นศูนย์ตลอดช่วง N ตัวแปร N เป็นตัวแปรที่จะต้องระวังมากเป็นพิเศษในการวิเคราะห์ความถี่ช่วงสั้น ถ้าค่าของ N ต่ำจะทำให้ความละเอียดในโดเมนความถี่หยาบมาก เพราะจะให้ผลที่ดีในโดเมนเวลา เพราะการเฉลี่ยถูกทำในช่วงสั้น ๆ เท่านั้น ในทางตรงกันข้ามถ้า N มีขนาดใหญ่จะให้ผลของความละเอียดเวลาที่แย่ในโดเมนเวลา แต่จะทำให้โดเมนความถี่มีความละเอียดสูงกว่า

2.10.1 การแปลงฟาสต์ฟูริเยร์ (Fast Fourier Transform)

โดยทั่วไปการแปลงฟูริเยร์ (Discrete Fourier Transform) เป็นการคำนวณที่ใช้เวลาก่อนข้างมาก ลำดับขั้นตอนในการคำนวณ DFT ให้เร็วขึ้นเรียกว่า ฟาสต์ฟูริเยร์ (Fast Fourier Transform หรือ FFT) โดยการแปลง FFT แบ่งได้เป็น 2 ชนิดใหญ่ ๆ คือ ชนิดลดทอนทางเวลา (Decimation In Time หรือ DIT) และชนิดลดทอนทางความถี่ (Decimation In Frequency หรือ DIF) สำหรับในส่วนนี้จะแสดงเฉพาะชนิดลดทอนทางเวลาซึ่งเกี่ยวกับในโครงงานนี้เท่านั้น

2.10.2 ขั้นตอนวิธีลดทอนทางเวลา

วิธีนี้เป็นการจัดแบ่งกลุ่มลำดับสัญญาณในโดเมนเวลา $X(n)$ ขนาด N จุดออกเป็น 2 ลำดับสัญญาณขนาด $N/2$ จุดเท่ากันคือ ลำดับคู่และลำดับคี่ โดยที่ลำดับคู่เกิดจากการเอาลำดับในตำแหน่งคู่มาเรียงกัน ที่เหลือเป็นลำดับคี่ ดังนั้นจะได้

$$\begin{aligned} x_c(m) &= x(2n) & ; m = 0, 1, 2, \dots, (N/2)-1 \\ x_o(m) &= x(2n+1) & ; m = 0, 1, 2, \dots, (N/2)-1 \end{aligned} \quad (2.4)$$

ถ้าให้ W_N เท่ากับ $\exp(-2j/N)$ จะทำให้การคำนวณ DFT ของลำดับ $x(n)$ ที่ยาว N จุดสามารถเขียนใหม่ได้เป็น

$$X(k) = \sum_{m=0}^{N/2-1} x_e(2m)W_N^{2km} + \sum_{m=0}^{N/2-1} x_o(2m+1)W_N^{(2m+1)k} \quad (2.5)$$

โดยที่

$$W_N^2 = \{\exp[j2\pi / N]^2\} = \exp[j2\pi / N / 2] = W_{N/2} \quad (2.6)$$

ซึ่ง $W_{N/2}$ เป็นค่า W ลำดับความยาว $N/2$ จุด สมการ 2.5 สามารถเขียนใหม่ได้เป็น

$$X(k) = \sum_{m=0}^{N/2-1} x_e(m)W_{N/2}^{km} + \sum_{m=0}^{N/2-1} x_o(m)W_{N/2}^{km} \quad (2.7)$$

การนำผลการแปลง DFT ขนาด 2 จุด จำนวน $N/2$ ภาคมารวมกัน เพื่อให้เป็นการคำนวณ DFT ขนาด N จุด จะต้องมียกเงื่อนไขที่ถูกต้องด้วย จากสมการ 2.7 ถ้าเขียนให้อยู่ช่วง $0 \leq k \leq N/2 - 1$ สามารถเขียนใหม่ได้เป็น

$$X(k) = X_e(k) + (W_N^k) X_o(k) \quad ; 0 \leq k \leq N/2 - 1 \quad (2.8)$$

$$= X_e(k - N/2) + (W_N^k) X_o(k - N/2) \quad ; 0 \leq k \leq N/2 - 1$$

เทอม W_N^k เรียกว่า ตัวประกอบหมุน (Twiddle Factor) ซึ่งใช้ร่วมกันกับ DFT ขนาด 2 จุด หรือขนาด $N/2$ จุด ในการนำมาประกอบเป็น DFT ขนาด N จุด ได้เหมือนเดิม และจากความสัมพันธ์

$$(W_N)^{k-N/2} = -(W_N)^k$$

จะได้

$$X(k) = X_1(k) + (W_N)^k X_2(k) \quad ; 0 \leq k \leq N/2 - 1 \quad (2.9)$$

$$= X_1(k - N/2) + (W_N)^{k-N/2} X_2(k - N/2) \quad ; N/2 \leq k \leq N - 1$$

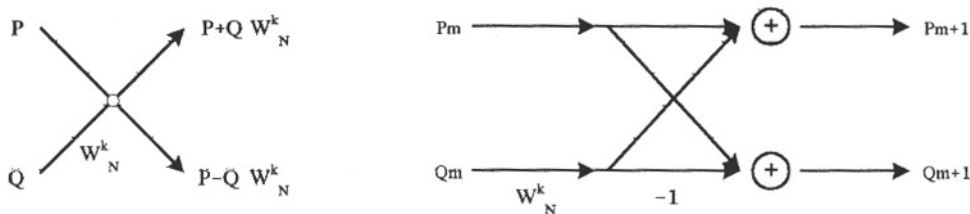
ผลจากสมการนี้สามารถนำไปใช้สร้างหน่วยคำนวณที่เรียกว่า หน่วยผีเสื้อ (Butterfly Unit) โดยมีข้อมูลเข้าคือ A และ B และข้อมูลออก คือ X และ Y เป็น

$$X = A + (W_N)^k B$$

$$Y = A - (W_N)^k B$$

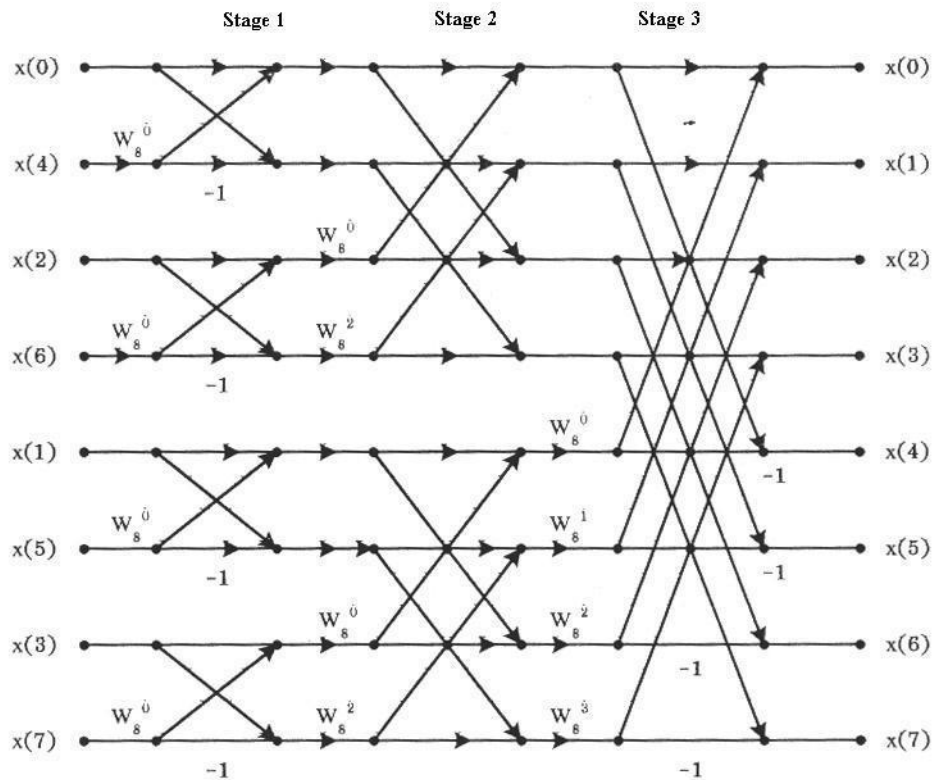
(2.10)

ซึ่งสามารถเขียนอธิบายแทนด้วยโฟลว์ชาร์ตดังแสดงในรูป 2.10



รูปที่ 2.10 แสดงหน่วยผีเสื้อของการคำนวณตามขั้นตอนวิธีลดทอนทางเวลา

สำหรับตัวอย่างการคำนวณ DFT โดยใช้ FFT แสดงดังรูป 2.11



รูปที่ 2.11 แสดงวิธีการของ FFT แบบลดทอนทางเวลา (DIT) สำหรับข้อมูลขนาดจุด

2.11 ความรู้เบื้องต้นเกี่ยวกับนิวรัลเน็ตเวิร์ค

ในช่วงระยะเวลา 8-9 ปีที่ผ่านมา ในต่างประเทศมีการตื่นตัวในการวิจัยและพัฒนาเกี่ยวกับโครงข่ายประสาทเทียม (Artificial Neural Networks : ANN) อย่างกว้างขวางทั้งทางด้านทฤษฎีและการประยุกต์ใช้งาน จนกระทั่งถึงปัจจุบันได้มีการประยุกต์นำนิวรัลเน็ตเวิร์คมาใช้ในการอุปกรณ์เครื่องใช้ต่าง ๆ มากขึ้น เช่น เครื่องมือหาปลา (Sonar) ที่มีความฉลาดมากขึ้น เช่น สามารถบอกได้ว่าฝูงปลาที่กำลังตรวจจับอยู่นั้นเป็นปลาชนิดใด จำนวนเท่าไร เครื่องโทรศัพท์แบบที่สามารถเรียกเลขหมายปลายทางให้อัตโนมัต (Voice Phone) เพียงกดหมายเลขของผู้ที่จะติดต่อเท่านั้น เครื่องอ่านตัวอักษร (OCR) ที่สามารถเปลี่ยนภาพตัวอักษรให้เป็นรหัสตัวอักษรแบบแอสกี ระบบนักบินอัตโนมัติ (Auto Pilot Aircraft) ระบบการคาดเดาอนาคตจากข้อมูลในอดีต (Forecasting, Prediction) ฯลฯ ซึ่งเครื่องมือและอุปกรณ์ต่าง ๆ ที่นำเอา ANN มาใช้ช่วยวิเคราะห์นั้น จะมีความฉลาดมากขึ้นและมีระบบความคิดที่มีการทำงานในลักษณะคล้ายกับมนุษย์

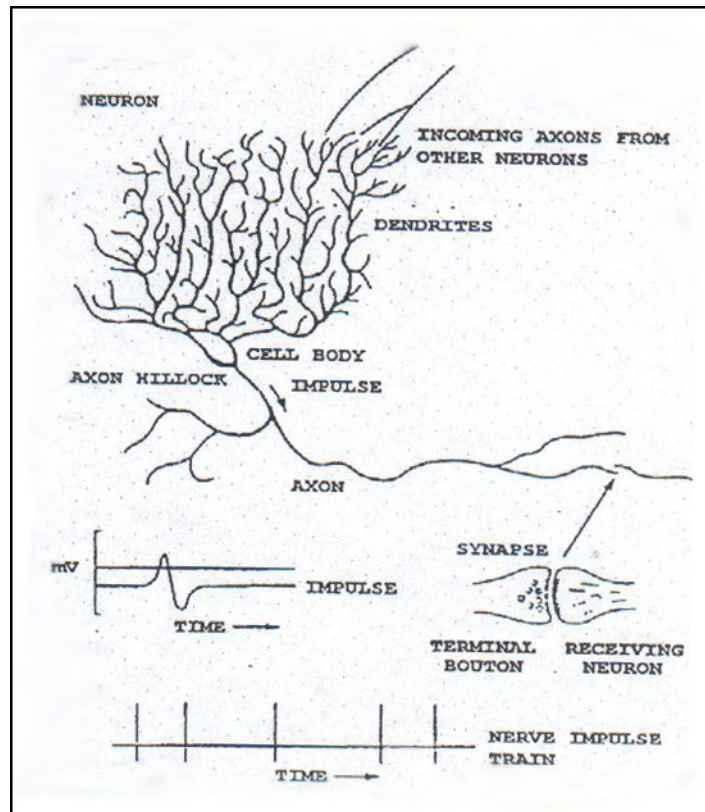
นิวรัลเน็ตเวิร์ค หมายถึง โครงข่ายใยประสาทที่เชื่อมต่อกันระหว่างเซลล์ประสาทจำนวนมากมายมหาศาล มีความสามารถประมวลผลสูงบรรจุอยู่ในสมอง สมองชีวภาพที่เป็นจุดศูนย์กลางการควบคุมกิจกรรมของการดำเนินชีวิตการวิจัยสร้างโครงข่ายประสาทเทียม (Artificial Neural Network) มีแนวคิดเลียนแบบการทำงานของสมองชีวภาพ โดยเรียนรู้และศึกษาการทำงานของสมอง

ชีวภาพเพื่อกำหนดแนวทางสำหรับการสร้างแบบจำลองขึ้นมา แล้วพยายามสมมติฐานลักษณะการทำงานโดยจำลองเป็นโมเดลคณิตศาสตร์ที่มีลักษณะเดียวกันแล้วดำเนินการคำนวณโดยใช้คอมพิวเตอร์

2.11.1 นิวรัลเน็ตเวิร์คชีวภาพ

ระบบคิดคำนึงของมนุษย์ มีโครงสร้างพื้นฐานจากเซลล์สมองที่เรียกว่านิวรัล (Neural) เรียงเป็นชั้นๆ อย่างซับซ้อนจำนวนมหาศาลมหาศาลประมาณหมื่นล้าน (10^{11}) นิวรัล และอาจมีจุดเชื่อมต่อภายในถึงพันล้านล้าน (10^{15}) จุด แต่ละนิวรัลจะมีคุณลักษณะแตกต่างกันไปโดยมีการทำงานคล้ายกัน คือ รับเข้า, ประมวลผล, ส่งออกสัญญาณไฟฟ้าเคมีผ่านไปยังนิวรัล ซึ่งจะส่งสื่อสารไปตามระบบของสมอง

จากภาพที่ 2.12 ส่วนแขนงที่ขยายแยกออกไปจากตัวเซลล์ต่อไปยังเซลล์อื่นๆเพื่อรับสัญญาณเรียกว่า เดนไดรต์ (Dendrites) จุดรับสัญญาณจากเซลล์อื่น เข้ามายังตัวเซลล์ จะผ่านมาทางจุดเชื่อมต่อที่เรียกว่า ซินแนปส์ (Synapse) ซึ่งแอกซอน (Axon) จะเป็นตัวส่งสัญญาณเอาที่พุท ออกไปยังนิวรอลอื่น จากผลการวิจัยพบว่า แต่ละนิวรอลจะเชื่อมต่อออกไปยังนิวรอลอื่นๆ ซึ่งแต่ละนิวรอลจะมีคุณสมบัติในการเพิ่ม ขยาย หรือลดทอนความเข้มของสัญญาณบางสัญญาณที่เข้ามาทางเดนไดรต์ของเซลล์ (ซึ่งมีแขนงมากมาย) อาจสามารถกระตุ้นตัวเซลล์ แต่บางสัญญาณก็อาจจะยับยั้งตัวเซลล์ เนื่องจากเซลล์ประสาทหนึ่งเซลล์ มีเดนไดรต์มาก ฉะนั้น สัญญาณกระตุ้นจากเดนไดรต์ ที่รับเข้ามาจากเซลล์ประสาทอื่นๆ จะถูกนำมารวมกันที่ตัวเซลล์ประสาทที่เซลล์ประสาทจะมีค่าเทรชโฮลด์ (Threshold) ค่าหนึ่งหากผลรวมของเซลล์ไฟฟ้าเคมี (Electrochemical) มีค่ามากกว่า เทรชโฮลด์เซลล์ประสาทก็จะส่งสัญญาณขนาดหนึ่งผ่านทางแอกซอน ไปยังนิวรอลอื่นๆ การจัดเรียงชั้น (Layer) และลักษณะการเชื่อมโยงระหว่างนิวรอลในสมองนั้น มีการจัดเรียงที่ซับซ้อน สอดคล้องกับหน้าที่การทำงานเฉพาะส่วน มีการเจริญเติบโตสัมพันธ์กับสิ่งแวดล้อม และมีการเรียนรู้ตลอดเวลา ซึ่งใช้เวลานานนับปี ดังนั้นจึงยากที่จะสร้างโมเดลขึ้นมา เพื่อเลียนแบบให้มีคุณลักษณะคล้ายสมองชีวภาพได้ทั้งหมด ผลงานที่ได้จากการทำวิจัยในปัจจุบันเป็นเพียงการจำลอง การเลียนแบบ และการทำงานเฉพาะบางส่วนของโครงข่ายประสาทมาใช้เฉพาะกับงานใดงานหนึ่ง ซึ่งมีการวิจัยลักษณะของโครงข่ายแบบต่างๆขึ้นมา โดยแต่ละแบบจะเหมาะกับงานประเภทหนึ่งๆเท่านั้น

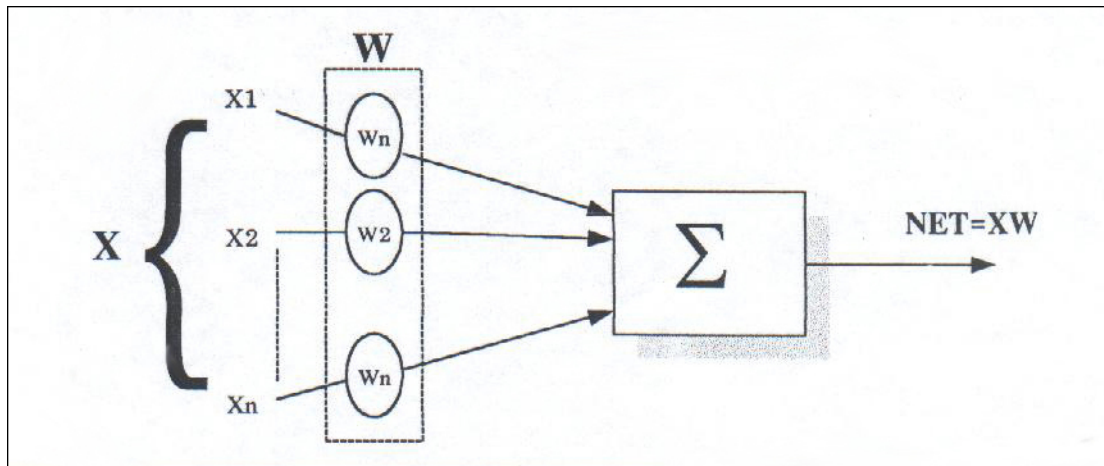


รูปที่ 2.12 แสดงโครงสร้างตัวอย่างของเซลล์ประสาทชีวภาพ

2.11.2 โครงข่ายประสาทเทียม (Artificial Neural Networks)

การออกแบบสร้างประสาทเทียมนั้น มีสมมติฐานขั้นแรกจากคุณสมบัติ ของระบบประสาทชีวภาพ ดังที่กล่าวมา กล่าวคือ ชุดรับสัญญาณข้อมูล อินพุตของเซลล์ประสาทหนึ่ง ได้จากสัญญาณเอาต์พุตของเซลล์ประสาทอื่นๆ ผ่านทางซินแนปส์และเดนไดรต์ ข้อมูลแต่ละค่าที่รับมาจะถูกลดขนาดด้วย ซินแนปติกส์ ซึ่งภายในประกอบด้วยสารเคมีประเภท K^+ , Ca^{++} , Na^+ , Cl^- ซึ่งจะมีลักษณะทางความนำพัลส์ (Pulse) สัญญาณไฟฟ้าเคมีที่แตกต่างกัน (James A. Freeman and David M. Skapura, 1991:8-9) ด้วยเหตุนี้ โมเดลประสาทเทียมที่สร้างขึ้น จะต้องมีการถ่วงน้ำหนักให้กับโมเดลก่อนที่จะนำเข้าสู่โมเดลประสาทเทียม จุดนี้เรียกว่า น้ำหนักซินแนปติกส์ ปริมาณของข้อมูลที่เข้าสู่นิวรอน จะถูกนำมารวมกัน และตัดสินใจด้วยระดับความสนใจของนิวรอน (Activation level) แล้วจะส่งเป็นเอาต์พุตออกที่แอกซอนไปยังนิวรอนอื่นๆ

จากภาพที่ 2.13 แสดงถึงโมเดลที่สร้างขึ้นโดยแนวความคิดจากเซลล์สมองชีวภาพ สัญญาณอินพุต คือ $X_1, X_2, \dots, X_n$ จะถูกป้อนเข้าไปยังนิวรอนที่สร้างขึ้น ซึ่งเปรียบเทียบกับสัญญาณที่ป้อนเข้ายังซินแนปส์ของนิวรอนชีวภาพ สัญญาณอินพุตนี้จะนำไปคูณกับค่าน้ำหนักซินแนปติกส์ที่มีค่าตั้งแต่ 0.00-1 (Weight : ค่าที่ใช้ถ่วงน้ำหนัก) $W_1, W_2, \dots, W_n$ ก่อนที่จะเข้าสู่บล็อกซัมเมชัน (Σ : Summation) ซึ่งค่าถ่วงน้ำหนักนี้ จะสอดคล้องกับค่าสเตรงท์ (Strength) ของจุดต่อซินแนปส์ชีวภาพ



รูปที่ 2.13 แสดงไดอะแกรมของนิวรอนที่สร้างขึ้น (Artificial Neuron)

แต่ละจุด (Single biological synaptic connection) บล็อกซิมเมชันนี้ ก็จะทำหน้าที่สอดคล้องคล้ายกับตัวเซลล์สมองชีวภาพ ผลรวมทางคณิตศาสตร์ของอินพุต และน้ำหนักจะได้เป็นเอาต์พุต เราเรียกว่า เน็ต (NET) ซึ่งเราจะรวมกันในรูปแบบของเวกเตอร์ได้ดังนี้

$$NET = X_1W_1 + X_2W_2 + \dots + X_nW_n \quad (2.11)$$

จะได้

$$NET = XW \quad (2.12)$$

2.11.3 ฟังก์ชันกระตุ้นความสนใจ (Activation Function)

เมื่อได้สัญญาณ NET แล้ว กระบวนการต่อมาที่นิวรอนต้องทำคือตัดสินใจ เราจึงต้องกำหนดฟังก์ชันการตัดสินใจ เพื่อใช้เป็นระดับของการตัดสินใจให้กับนิวรอน เพื่อให้ได้สัญญาณเอาต์พุตของนิวรอนออกมา ซึ่งเชื่อมต่อไปยังนิวรอนตัวอื่นๆเป็นโครงข่าย OUT ที่ได้จากอาจเป็น Simple linear function โดย

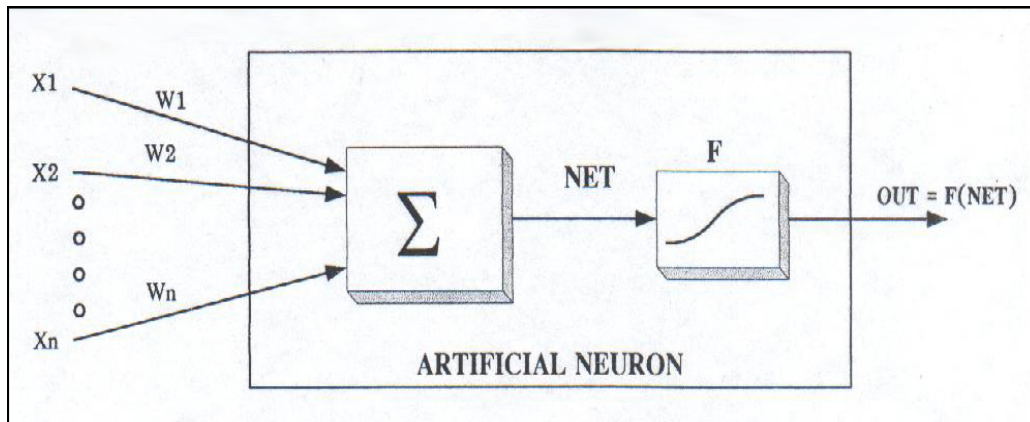
$$OUT = K[NET] \quad (2.13)$$

โดย K เป็นค่าคงที่ ที่เรียกว่า Threshold function

ตัวอย่างเช่น

$$\begin{aligned} OUT &= 1 && \text{ถ้า } NET > T \\ OUT &= 0 && \text{เมื่อเป็นกรณีอื่นๆ} \end{aligned} \quad (2.14)$$

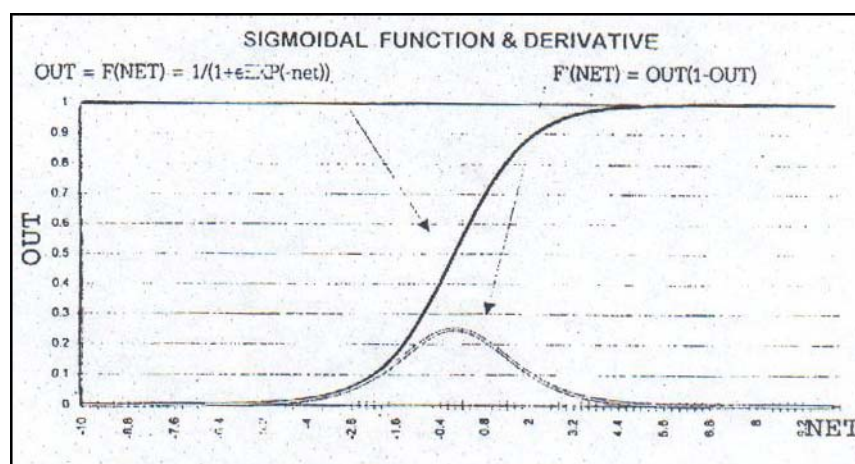
และ T เป็นค่าเทรสโกลงที่ หรืออาจเป็น Function อื่นๆ ที่เลียนแบบคุณสมบัติไม่เป็นเชิงเส้นของเซลล์ประสาทชีวภาพได้อย่างใกล้เคียงกว่า และใช้เป็นฟังก์ชันให้กับโครงข่ายทั่วไปได้



รูปที่ 2.14 แสดงโมเดลนิวรอลที่สร้างขึ้นร่วมกับ Activation Function

ในภาพที่ 2.14 บล็อก F จะได้รับผลที่ได้จาก NET มาสร้างเป็นสัญญาณเอาต์พุตที่ OUT โดยกระบวนการภายในบล็อก F จะบีบช่วงของ OUT ให้อยู่ในขอบเขตจำกัด ตามต้องการ ดังนั้น ค่า OUT จะมีค่าไม่ต่ำกว่าช่วงที่กำหนดโดยค่าของ NET เราเรียกบล็อก F นี้ว่าสแควชชิงฟังก์ชัน (Squashing function) และโดยทั่วไป สแควชชิงฟังก์ชันที่ใช้เป็นแบบลอจิสติกฟังก์ชัน หรือซิกมอยด์ (Logistic function or “Sigmoid”) ซึ่งมีรูปร่างคล้ายตัว S โดยเขียนเป็นสมการคณิตศาสตร์ได้ดังนี้คือ

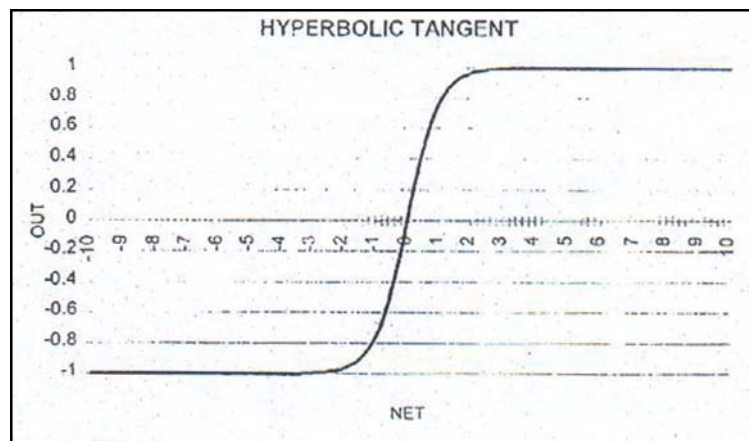
$$F(x) = \frac{1}{(1 + e^{-x})} \quad (2.15)$$



รูปที่ 2.15 แสดงกราฟที่ได้จากสมการซิกมอยด์ลอจิสติกฟังก์ชัน (Sigmoidal logistic function)

ลักษณะของเทรโซลฟังก์ชันมีลักษณะเป็น Non-linear function เช่น S-Curve เราจะได้ค่าเอาต์พุต ที่มีความไวต่อสัญญาณอินพุตที่มีขนาดเล็กๆ และเฉลี่ยต่อสัญญาณแรงๆ ซึ่งสัญญาณอ่อนๆ ไปทางบวกเพียงเล็กน้อยก็จะทำให้ OUT ใกล้เคียง “1” กระตุ้นหรือสัญญาณอ่อนๆ ทางลบเพียงเล็กน้อย ก็จะทำให้ Output ใกล้เคียง “0” (ยับยั้ง) ขณะที่สัญญาณแรงๆ ทางบวกก็ยังคงให้ Output ใกล้เคียง “1” และสัญญาณทางลบแรงๆ ก็คงให้ Output ใกล้เคียง “0” เช่น คุณลักษณะแบบนี้ เป็นแบบ NON-LINEAR GAIN ซึ่งคลอสส์เบอร์ก (Grossberg, 1973) พบว่า คุณลักษณะที่เป็น Non-linear gain นี้สามารถแก้ปัญหา Noise-saturation dilemma ได้ และทำให้นิวรอลเทียมที่สร้างขึ้นสามารถทำงานกับขนาดของอินพุตได้กว้างมากขึ้น

ยังมีฟังก์ชันอื่นๆ อีกคือ ไฮเปอร์โบลิค แทนเจนต์ (Hyperbolic tangent) มันจะมีลักษณะคล้ายกับ Logistic function และนิยมใช้บ่อยๆ ในการสร้างโมเดลคณิตศาสตร์ การกระตุ้นเร้าความสนใจของเซลล์สมองเทียม ซึ่งมีคุณสมบัติคล้ายชีวภาพของเซลล์สมองคือ $OUT = \tanh(X)$



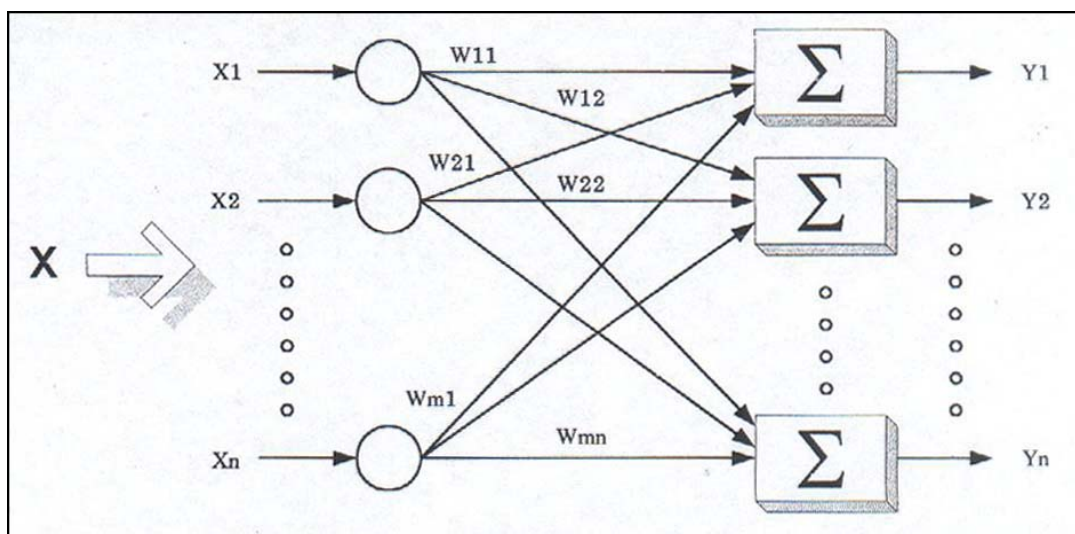
รูปที่ 2.16 แสดง Hyperbolic Tangent Function

จากภาพที่ 2.16 ส่วนที่เหมือนกับ ซิกมอยด์ ลอจิสติก ฟังก์ชัน คือมีลักษณะเป็น S แต่เนื่องจากมันจะมีความสมมาตร จึงให้ OUTPUT อยู่ระหว่าง “-1” ถึง “1” OUTPUT จะเป็น “0” เมื่อ NET เป็น “0” OUTPUT เข้าใกล้ “1” เมื่ออินพุตไปทางบวก และเข้าใกล้ “-1” เมื่ออินพุตไปทางลบ และเข้าใกล้ “-1” เมื่ออินพุตมีทิศทางไปทางลบ

2.11.4 โครงข่ายประสาทเทียมแบบชั้นเดียว (Single Layer Artificial Neural Networks)

ที่กล่าวมาจนถึงจุดนี้ เป็นการกล่าวถึงหลักการและเหตุผลในการสร้างเซลล์ประสาทเทียมเพียงหนึ่งเซลล์ โดยใช้แนวความคิดจากเซลล์ประสาทชีวภาพ การจะนำเซลล์ประสาทเทียมมาใช้งานได้นั้น ต้องใช้เซลล์ประสาทเทียมที่มีคุณลักษณะต่าง ๆ กัน (ค่า Weight จะทำให้คุณสมบัติของเซลล์ประสาทเทียม แต่ละเซลล์มีคุณสมบัติแตกต่างกันออกไป) มาเชื่อมโยงเป็นโครงข่ายในลักษณะ

เดียวกับเซลล์สมองชีวภาพเสียก่อน ซึ่งลักษณะการเชื่อมโยงมีหลายชนิด แต่ละชนิดก็มีคุณลักษณะเด่นที่แตกต่างกันไป



รูปที่ 2.17 แสดงลักษณะโครงข่ายประสาทเทียมแบบชั้นเดียว (Single-Layer Neural Network)

จากรูป 2.17 เป็นโครงข่ายประสาทเทียมแบบชั้นเดียว ที่ประกอบด้วยเซลล์ประสาทเทียมง่ายๆหลายๆชุด ความสามารถในการคำนวณของโครงข่ายประสาทเทียม ได้มาจากลักษณะการเชื่อมต่อ เป็นโครงข่ายประสาทเทียมโครงข่ายง่ายๆ เป็นกลุ่มโมดูลประสาทเทียมที่เชื่อมต่อกันเป็นชั้นๆ (Layer) ในภาพที่ 2.18 เป็นโครงข่ายประสาทเทียมแบบชั้นเดียว (Single layer) ที่ประกอบด้วยเอาต์พุตเลเยอร์ (กลุ่มของบล็อกลักษณะที่อยู่มุมขวามือ) และอินพุตเลเยอร์ (วงกลมทางซ้ายมือ) โดยไม่พิจารณาอินพุตเลเยอร์ว่าเป็นนิวรอลเลเยอร์ เนื่องจากอินพุตเลเยอร์จะทำหน้าที่เชื่อมต่ออินพุตที่รับมา และส่งออกไปให้ยังแต่ละอินพุตนิวรอลเลเยอร์ (ในที่นี้คือ Output layer) ในชั้นถัดไป โดยแต่ละอินพุตจะถูกคูณโดยค่าน้ำหนักเฉพาะแต่ละอินพุต โครงข่ายประสาทเทียมที่สร้างขึ้นในขั้นแรกไม่ซับซ้อน โดยแต่ละนิวรอลจะได้เอาต์พุตจาก

$$\text{OUT} = \text{Logistic Function} \text{ คูณ (ผลรวมของ Input X กับ Weight)}$$

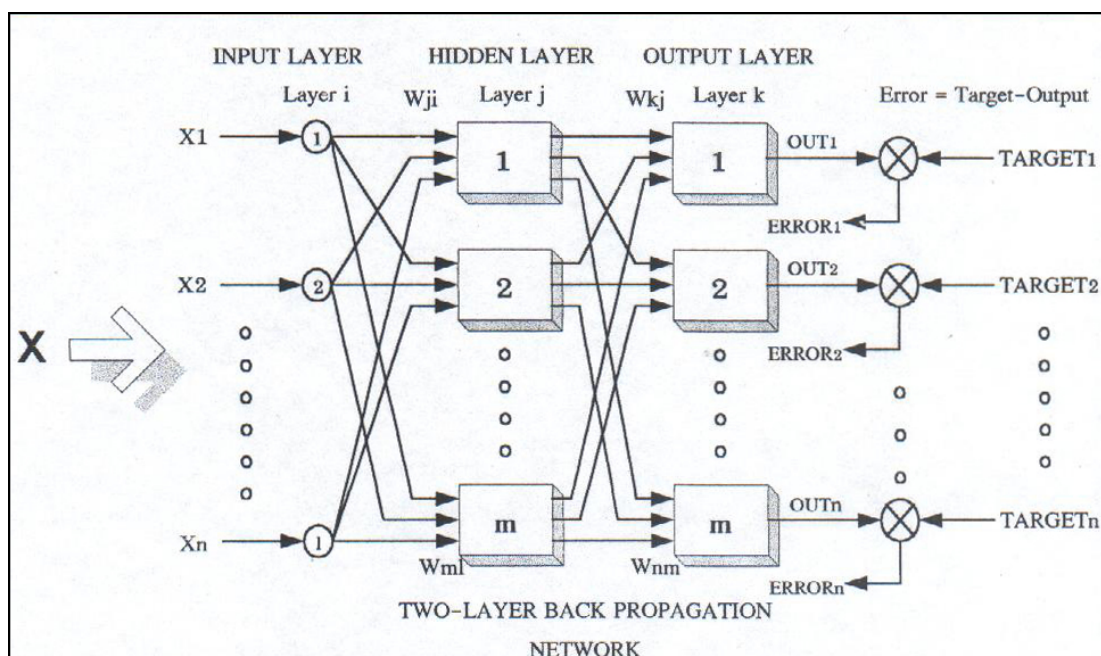
หรือ
$$\text{OUT} = F(\text{NET}) \quad (2.16)$$

อย่างไรก็ดีลักษณะการเชื่อมโยงระหว่างโครงข่ายไม่ได้มีแบบเดียว การเชื่อมโยงระหว่างเลเยอร์อาจมีการเชื่อมโยงย้อนกลับมาจากอินพุตเลเยอร์อีก ซึ่งโครงข่ายประสาทชีวภาพก็มีลักษณะดังกล่าวเช่นกัน สำหรับค่า Weight ในภาพที่ 2.18 มีวิธีการพิจารณาในรูปของ เมตริกน้ำหนัก (Weight matrix) ซึ่งหากโครงข่ายมีหลายชั้น จะช่วยให้ระบุค่าน้ำหนักได้ง่ายขึ้น และเพื่อหลีกเลี่ยงความสับสนจะกำหนดเป็นโดเมนชั้น (Dimensions) ของเมตริก โดยให้ m แทนจำนวนแถว หรือจำนวนของอินพุต

และ n แทนจำนวนของนิวรอน ที่สร้างขึ้น ตัวอย่างเช่น น้ำหนักที่เชื่อมระหว่างอินพุตตัวที่ 4 กับ นิวรอนตัวที่ 2 คือ $W_{4,2}$

2.11.5 โครงข่ายประสาทเทียมแบบหลายชั้น (Multilayer Artificial Neural Networks)

โครงข่ายที่ซับซ้อนจะมีความสามารถในการคำนวณที่ดีขึ้นมันจะเป็นโครงข่ายที่มีโครงสร้างเป็นจินตนาการที่น่าเป็นไปได้โดยการจัดการเชื่อมโยงนิวรอน มีโครงสร้างเป็นชั้นๆ คล้ายส่วนหนึ่งของสมอง และมีการพัฒนาอัลกอริทึมเกี่ยวกับการฝึกสอนให้โครงข่ายแบบหลายชั้นทำงานได้ ตามความต้องการแล้วเมื่อไม่นานมานี้ โครงข่ายแบบหลายชั้นอาจจะสร้างจาก กลุ่มของโครงข่ายแบบชั้นเดียวเอาต์พุตของ Layer หนึ่ง จะใช้เป็นอินพุตของ Layer ถัดไป ในภาพที่ 2.18 แสดงเน็ตเวิร์กที่มีการเชื่อมต่อแบบสองชั้น



รูปที่ 2.18 แสดงไดอะแกรมของ Backpropagation Neural Networks แบบสองชั้น

ภาพที่ 2.18 โครงข่ายประสาทเทียมแบบหลายชั้น ที่ต่อเชื่อมโยงแบบเต็มชั้น ในโครงข่ายแบบหลายชั้นมีการเรียกชื่อชั้นต่างๆ ดังนี้ คือ ชั้นที่ต่อโดยตรงกับอินพุต เรียกว่า อินพุตเลเยอร์ (Input layer) ชั้นนี้จะไม่มีการคำนวณ แต่จะทำหน้าที่ต่อเชื่อมข้อมูลไปยังชั้นถัดไป ชั้นที่อยู่ท้ายสุดทางขวามือเรียกว่า เอาต์พุตเลเยอร์ (Output layer) เป็นชั้นที่โครงข่ายจะให้ผลลัพธ์ ส่วนชั้นที่อยู่ระหว่างอินพุตเลเยอร์ และเอาต์พุตเลเยอร์ จะมีกี่ชั้นก็ตามจะเรียกว่า ฮิดเดนเลเยอร์ (Hidden layer) หากฮิดเดนเลเยอร์ มีหลายๆชั้น ก็จะมีการตั้งชื่อเฉพาะลงไปให้กับแต่ละชั้น

2.11.6 ฟังก์ชันกระตุ้นความสนใจแบบไม่เป็นเชิงเส้น (The Nonlinear Activation Function)

การนำเอาที่พุดของเลเยอร์หนึ่ง มาเชื่อมกับอินพุตของเลเยอร์ชั้นถัดไป โดยผ่านฟังก์ชันกระตุ้นความสนใจแบบไม่เป็นเชิงเส้น จะทำให้โครงข่ายมีความสามารถในการคำนวณเพิ่มขึ้น (หากไม่ผ่านฟังก์ชันดังกล่าวความสามารถการคำนวณจะไม่เพิ่มขึ้น และจะมีความสามารถไม่แตกต่างไปจาก Single layer networks) และสามารถกำหนดขอบเขตของเอาต์พุตให้อยู่ในช่วงที่ต้องการได้

2.11.7 การฝึกสอนให้กับโครงข่ายประสาทเทียม (Training of Artificial Neural Networks)

ค่าน้ำหนัก มีความสัมพันธ์กับอะไร เปลี่ยนแปลงอย่างไรนั้น ก็เช่นเดียวกับเด็กที่คลอดออกมาที่มีสมองแล้ว แต่สมองยังไม่เจริญเติบโตเพียงพอ และยังไม่ได้รับการฝึกสอนและเรียนรู้ เด็กจึงไม่สามารถทำกิจกรรมใดๆด้วยตนเอง เว้นแต่กิจกรรมที่ธรรมชาติสร้างมาพร้อมกับการกำเนิดที่เรียกว่า “สัญชาตญาณ” ซึ่งธรรมชาติได้คุณลักษณะบางอย่างให้เซลล์สมองบางส่วนตั้งแต่ทารกเจริญเติบโตอยู่ในครรภ์มารดา เช่น ระบบควบคุมการหายใจ, ความรู้สึก, การเรียกร้องเมื่อหิว, การตอบสนองต่อสิ่งเร้า ฯลฯ เด็กจะพัฒนาการเรียนรู้ไปตามขั้นตอน หลังจากนั้นสมองของเข่าจะได้รับ การฝึกสอน และเจริญเติบโตไปพร้อมๆกัน เซลล์สมองจะได้รับการปรับคุณลักษณะสอดคล้องกับการฝึกสอน และจะเจริญเป็นโครงข่ายสอดคล้องกัน

โครงข่ายประสาทเทียมที่สร้างขึ้นมีลักษณะเช่นเดียวกัน คือ เมื่อสร้างเสร็จ แต่ละเซลล์ประสาทที่สร้างขึ้นมานั้นจะยังไม่มีคุณลักษณะใดเลย เนื่องจากยังไม่มีกำหนดค่าซินแนปติกส์ที่เหมาะสมกับงานที่ต้องการให้กับมัน จึงต้องมีการฝึกสอนเพื่อให้เน็ตเวิร์คที่สร้างขึ้นมีลักษณะตามที่ต้องการ การฝึกสอนของโครงข่ายประสาทเทียม จะกระทำโดยการปรับเปลี่ยนค่าน้ำหนักซินแนปติกส์ เพื่อให้โครงข่ายจดจำ แพตเทิร์น ความสัมพันธ์ระหว่างอินพุตกับเอาต์พุตได้ โดยในขั้นแรกอาจกำหนดเป็นค่าสุ่มใดๆ (Random weight) ก่อนแล้วถึงปรับเปลี่ยนน้ำหนักไปตามอัลกอริทึมสมมติฐานหลายๆรอบ จนกว่าจะได้เอาต์พุตที่ต้องการ ในแง่ของไขความผิดพลาดที่ยอมรับได้

2.11.8 วัตถุประสงค์ของการเทรนนิ่ง (Objective of Training)

เนื่องจากค่าน้ำหนักที่ให้เป็นค่าสุ่มใดๆ โครงข่ายจึงไม่แสดงคุณลักษณะใดออกมา การฝึกสอน (Training) ให้โครงข่ายก็คือ การปรับค่าน้ำหนักทุกๆจุด ให้สอดคล้องกับอินพุตหลายๆแบบ เพื่อให้ได้เอาต์พุตตามต้องการนั่นเอง การฝึกสอนโครงข่ายจะต้องบรรลุถึงกระบวนการเข้าใจพื้นฐานเสียก่อน คือ การเรียนรู้ในโครงข่ายประสาทเทียมนั้นมีขีดจำกัด ปัญหาต่างๆ ผู้ใช้คงต้องแก้ไขมันก่อน แล้วนำผลนั้นไปอ้างอิงสำหรับการปรับปรุงค่าน้ำหนัก หลังจากปรับน้ำหนักจนได้ค่าผิดพลาดที่เอาต์พุตเทียบกับเป้าหมายน้อยลงเป็นที่พอใจแล้ว โครงข่ายประสาทเทียมนั้นก็พร้อมที่จะวิเคราะห์อินพุต และให้เอาต์พุตตามลักษณะตัวอย่างที่มันเคยเรียนรู้มา การเรียนรู้จะมีการปรับน้ำหนักหลายๆรอบ จนค่าน้ำหนักสอดคล้องกับตัวอย่างหลายๆตัวอย่าง และให้เอาต์พุตตามต้องการ

พบว่าโครงข่ายได้ตัวอย่างสำหรับการเทรนนิ่งมากๆ โครงข่ายก็จะมีความแม่นยำสูงขึ้น แต่ก็ใช้เวลาในการเทรนนิ่งเพิ่มขึ้นเช่นกัน หากพิจารณาต่อไปจะพบว่า โครงข่ายประสาทเทียมที่สร้างขึ้นจะมีพฤติกรรมคล้ายกับระบบการเรียนรู้ของมนุษย์มาก เป็นเพราะมีต้นแบบมาจากระบบประสาทชีวภาพนั่นเอง

2.11.9 การเทรนนิ่งแบบควบคุม (Supervised Training)

เทรนนิ่งอัลกอริทึมถูกจัดเป็น 2 ประเภท คือ แบบควบคุม (Supervised training) และแบบอิสระ (Unsupervised training) โดยการเทรนนิ่งแบบควบคุม จะต้องการคู่ของเทรนนิ่งระหว่างอินพุตกับเป้าหมายที่ต้องการที่เรียกว่า เทรนนิ่งแพร์ (Training pairs) โครงข่ายจะถูกเทรนไปตามจำนวนของคู่ที่เทรนนิ่ง (จำนวนคู่ของ Input กับ Output ที่ต้องการให้โครงข่ายรู้จัก) เอาท์พุทที่คำนวณได้จากโครงข่ายจะถูกเปรียบเทียบกับความสอดคล้องกับเป้าหมาย ค่าผิดพลาดที่เกิดขึ้นจะถูกป้อนกลับไปยังโครงข่าย และเปลี่ยนแปลงค่าน้ำหนักให้สอดคล้องกับอัลกอริทึม ที่ทำให้แนวโน้มของค่าผิดพลาดที่เกิดขึ้นระหว่างเอาท์พุทกับเป้าหมายโดยเฉลี่ยมีค่าลดต่ำลง ตัวอย่างการเทรนนิ่งแบบนี้ ได้แก่ การเทรนนิ่งแบบแพร่กลับ (Back propagation)

2.11.10 การเทรนนิ่งแบบอิสระ (Unsupervised Training)

ถึงแม้ว่า อัลกอริทึมแบบควบคุม (Supervised training) สามารถจะประยุกต์ใช้เพื่อปรับคุณลักษณะของโครงข่ายได้สำเร็จ แต่ก็ยังมีข้อวิจารณ์อยู่ คือ มันเป็นไปอย่างแบบชีวภาพไม่ได้ และยากที่จะเชื่อได้ว่า กลไกการเทรนนิ่งของสมองจะต้องการการเปรียบเทียบระหว่างคนที่ต้องการกับเอาท์พุทจริง โดยการบวนการป้อนกลับไปแก้ไขคุณลักษณะของโครงข่าย และถ้าสมมติว่า ถ้าสมองมีกลไกเช่นนี้ ต้องมีเอาท์พุทที่ต้องการเพื่อนำมาเป็นเป้าหมายตลอดเวลา และจะเอามาจากที่ใด สรุปคือต้องมีผู้คิดเป้าหมายให้กับโครงข่ายก่อน โครงข่ายไม่สามารถคิดและปรับคุณลักษณะได้ก่อนด้วยตนเอง ในทางตรงกันข้ามหากพิจารณาทารกแรกเกิดสมองของเขาสามารถจัดระบบเองได้อย่างไร การเทรนนิ่งแบบระบบของสมอง จนกระทั่งมีการพัฒนาการเทรนนิ่งแบบอิสระนี้ขึ้นราวปี 1984 โดย โคโฮเนน (Kohonen) และบุคคลอื่นๆ โดยได้เสนอแนวคิดที่เป็นการเทรนนิ่งแบบไม่ต้องการเป้าหมาย ไม่มีการตัดสินใจด้วยเหตุผลในอุดมคติมาก่อน ชุดของการเทรนนิ่ง จะมีเพียงอินพุทเวกเตอร์เท่านั้น เทรนนิ่งอัลกอริทึมจะเปลี่ยนแปลงค่าน้ำหนักของโครงข่าย เพื่อสร้างเอาท์พุทที่มีความมั่นคง ยกตัวอย่างเช่น หากให้โครงข่ายรู้จำภาพหน้าคนหนึ่ง หากภาพหน้าคนคนนั้นเปลี่ยนแปลงไปเล็กน้อย (Image อาจมี Noise รวมอยู่บ้าง) โครงข่ายนั้นก็ยังสามารถบอกได้ว่า คนคนนั้นเป็นคนเดิมเป็นต้น การเทรนนิ่งจะไม่มี การตัดสินใจมาก่อน ไม่มีการกำหนดแบบเอาท์พุทมาก่อน (อาจกล่าวได้ว่าแบบเอาท์พุทจะถูกกำหนดโดยอินพุทเวกเตอร์นั่นเอง) ดังนั้น เอาท์พุทของโครงข่ายก็เช่นกัน ส่วนใหญ่จะถูกแปรรูปซึ่งจะเข้าใจได้ภายหลังกระบวนการเทรนนิ่ง ดังนั้นจึงไม่สามารถ

แก้ปัญหาที่เคร่งครัดสำคัญได้ แต่มักนิยมใช้โครงข่ายแบบนี้กับงานง่ายๆ ประเภทการเปรียบเทียบเอกลักษณ์, รูปแบบที่สัมพันธ์กันระหว่างอินพุต-เอาต์พุต ที่ถูกกำหนดโดยโครงข่าย

2.11.11 วิธีการแก้ปัญหาคือการฝึกสอน (Training Algorithm)

ส่วนใหญ่แล้วทุกวันนี้ การแก้ปัญหาคือการฝึกสอนของโครงข่ายค่อยๆ พัฒนาก้าวหน้าขึ้นจากแนวความคิดของ ดี โอ เฮบบ์ (ปี 1961) เขาได้เสนอโมเดลของ การเทรนนิ่งแบบอิสระ (Unsupervised training) ในแบบซินแนปติกส์ตรงที่หรือน้ำหนัก ซึ่งจะเพิ่มขึ้น ถ้าทั้งแหล่งกำเนิด (Input Source) และจุดหมายปลายทาง (Destination) ของนิวรอนได้รับการสนใจ กรณีนี้ถ้ามีการใช้งานทางเส้นนี้บ่อยๆ ก็จะทำให้ซินแนปติกส์ตรงที่ (หรือน้ำหนัก) แข็งแรงขึ้น (เซลล์สมองที่ใช้งานมากบ่อยๆ ก็จะทำให้ ซินแนปส์ใหญ่ขึ้น การส่งผ่านข้อมูลพัลส์ไฟฟ้าทำได้ดีขึ้น ทำให้มีประสิทธิภาพดีขึ้น เช่น สามารถคิด หรือจดจำได้เร็ว และดีขึ้น) โครงข่ายประสาทเทียมนี้ ใช้การเรียนรู้แบบเฮบบ์ (Hebbian learning) จะเพิ่มค่าน้ำหนักของโครงข่ายอย่างสอดคล้องกับผลคูณของระดับความสนใจของแหล่งกำเนิด และจุดหมายของนิวรอนตามสมการดังนี้

$$W_{ij}(n+1) = W_{ij}(n) + \alpha OUT_i OUT_j \quad (2.17)$$

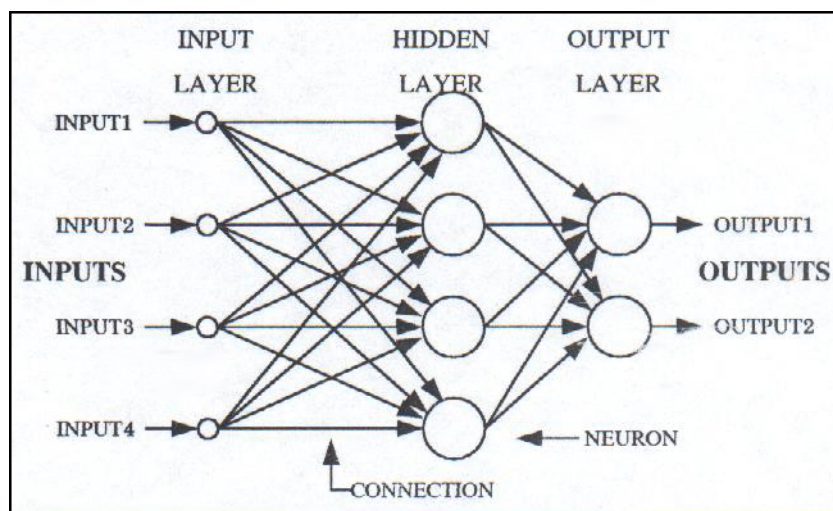
โดย	$W_{ij}(n)$	คือค่าน้ำหนัก จากนิวรอน I ไปยังนิวรอน j ก่อนปรับปรุงค่า
	$W_{ij}(n+1)$	คือค่าน้ำหนัก จากนิวรอน I ไปยังนิวรอน j หลังปรับปรุงค่า
	α	คือค่าคงที่ของการเรียนรู้ (Learning rate coefficient)
	OUT_i	คือเอาต์พุตของนิวรอน I และเป็นอินพุตของนิวรอน j
	OUT_j	คือเอาต์พุตของนิวรอน j

โครงข่ายที่มีลักษณะการเทรนนิ่งแบบเฮบบ์นั้น เป็นผลมาจากการพัฒนามาแล้วกว่า 20-30 ปี โดยเฉพาะงานของโรเซนเบลทท์ (Rosenblatt:1962), วิโดว์ (Widrow:1959), วิโดว์และฮอล์ฟ (Widrow&Holf:1960) และอีกหลายๆคนที่พยายามพัฒนาระบบการเทรนนิ่งแบบควบคุม ที่สร้างโครงข่ายที่สามารถเรียนรู้แบบของอินพุตได้อย่างกว้างขวาง และมีอัตราการเรียนรู้สูง ที่บรรลุผลได้จากหลักการพื้นฐานของการเรียนรู้ หรือเทรนนิ่งให้กับโครงข่ายเช่น Perceptrons, Hopfieldnets, Backpropagation Networks และ Counter propagation.

2.11.12 การประยุกต์ใช้นิวรัลเน็ตเวิร์กกับปริญาณิพนธ์

จากการศึกษาพฤติกรรม การสื่อสารของมนุษย์พบว่า การเรียนรู้ภาษาต้องเรียนรู้จากผู้อื่น ภาษาไม่ใช่สิ่งที่เป็นสัญชาตญาณ ดังนั้นจึงต้องเรียนรู้จากภาษาที่สังคมนั้นกำหนดขึ้น ซึ่งเป็นแพต

เทอร์นของภาษาที่บรรพบุรุษได้สร้างขึ้นสั่งสมกันมาก่อน จากเหตุผลดังกล่าว พบว่าสอดคล้องกับ อัลกอริทึม การเทรนนิ่งแบบควบคุม (Supervised training) ผู้ทำจึงเลือกใช้วิธี Backpropagation มัลติเลเยอร์เป็นแบบ 2 เลเยอร์ แสดงดังภาพที่ 2.19

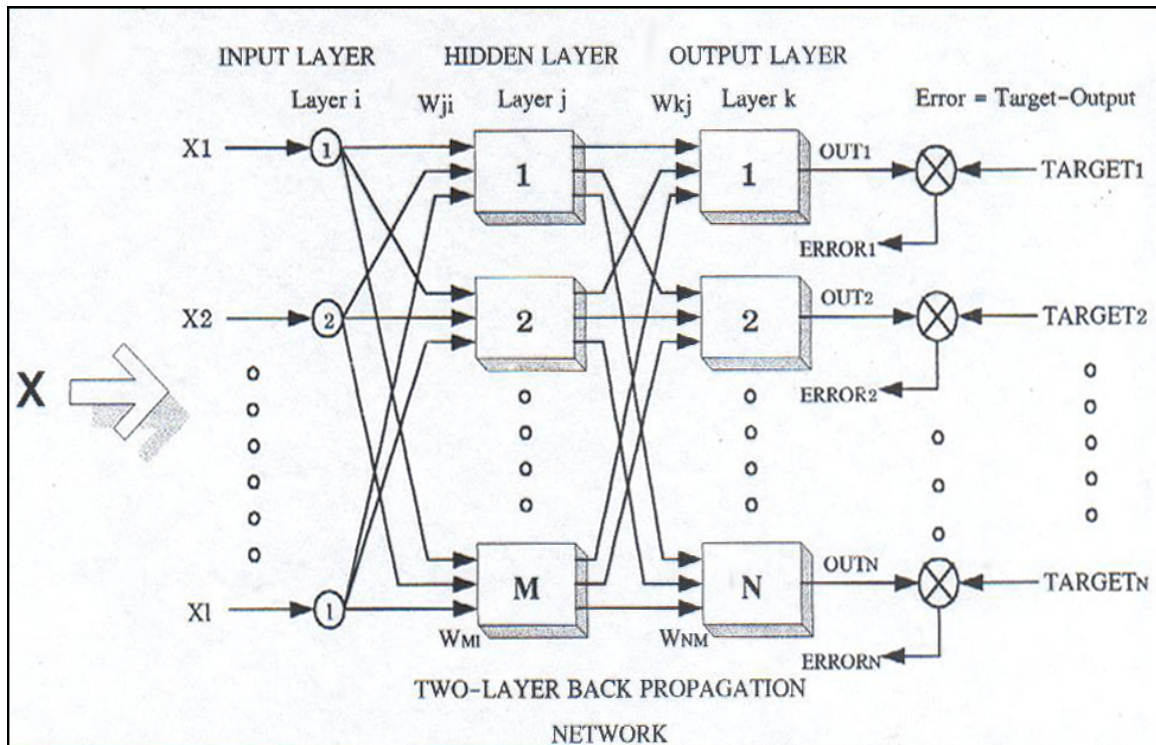


รูปที่ 2.19 แสดงไดอะแกรมของ Two Layer Feed - Forward

พิจารณาภาพที่ 2.19 ประกอบ จะเห็นว่า ประกอบด้วย input array, hidden layer และ output layer ในส่วนของ Input array จะไม่ถือเป็น layer เพราะไม่มีการคำนวณใดๆ แต่มันจะทำหน้าที่เชื่อมโยงข้อมูลให้กับ Hidden layer ทุกๆนิวรอน output ของ hidden layer ก็จะถูกเชื่อมโยงไปเป็น input ให้กับ output layer ทุกอินพุตของนิวรอนทุกตัวจะมีค่า Weight ประจำตัว ค่า Weight นี้ทำให้ระดับของอินพุตผ่านเข้าไปภายในนิวรอนแต่ละตัวไม่เท่ากัน ซึ่งเป็นตัวกำหนดคุณลักษณะสมบัติของนิวรอนแต่ละตัวในเน็ตเวิร์ค วิธีการกำหนดคุณสมบัติดังกล่าว ทำได้โดยการเทรนนิ่งโดยนัยชั้นแรก จะสมมติว่า ค่าแบบสุ่มน้อยๆให้ (ช่วงที่ใช้คือตั้งแต่ -0.5 ถึง +0.5) แล้วจึงค่อยๆปรับเปลี่ยนค่าน้ำหนักในแต่ละ layer ให้สอดคล้องกับคุณสมบัติที่ต้องการ

2.11.13 กระบวนการเทรนนิ่ง

กระบวนการเทรนนิ่ง คือ กระบวนการปรับคุณลักษณะของนิวรอนเน็ตเวิร์ค ให้มีคุณสมบัติตามที่ต้องการ การเทรนนิ่งของอัลกอริทึม Backpropagation เป็นแบบ Supervised Training แบ่งเป็น 2 ส่วนคือ Forward pass และ Reversed pass การเทรนนิ่งจะกระทำทั้ง Forward pass และ Reversed pass สลับกันไป จนกว่าค่าผิดพลาดระหว่าง Output กับ Target จะมีค่าต่ำกว่าจุดที่ต้องการ



รูปที่ 2.20 แสดงไดอะแกรมเชื่อมโยงระหว่างชั้นของนิวรอลแบบ 2 ชั้น
ของ Backpropagation Network

FORWARD PASS

จากภาพ 2.20 อินพุต X ตั้งแต่ X_1 ถึง X_l จะถูกคูณด้วย Weight matrix ผลรวมของอินพุตเมื่อคูณกับค่า Weight ของแต่ละนิวรอลเรียกว่า เน็ต (NET) เขียนเป็นสมการได้คือ

$$NET_j = X_1 W_{j1} + X_2 W_{j2} + \dots + X_l W_{jl} \quad (2.18)$$

หรือ

$$NET_j = \sum_{i=1}^l X_i W_{ij} \quad (2.19)$$

- โดย
- X คือ ค่าที่รับเข้ามาของ Input node
 - W คือ ค่าน้ำหนักประจำตัวของแต่ละอินพุตโหนด
 - I คือ หมายเลขโหนดโดยเริ่มที่โหนด 1 ถึง โหนด I
 - J คือ หมายเลขนิวรอลใน Hidden layer ตั้งแต่ นิวรอลที่ 1 ถึง M
 - K คือ หมายเลขนิวรอลใน Output layer ตั้งแต่ นิวรอลที่ 1 ถึง N

และทำนองเดียวกัน

$$NET_k = \sum_{j=1}^M OP_j W_{kj} \quad (2.20)$$

เมื่อได้ค่า NET ของแต่ละนิวรอนแล้ว จะปรับให้ได้ Output อยู่ในช่วงที่ต้องการเป็นค่าที่จะนำไปยับยั้ง หรือกระตุ้นนิวรอนตัวถัดไปโดย Squashing Function ที่ใช้เป็น Logistic Function หรือ Sigmoid จะได้ตามสมการ 2.21 และ 2.22

$$OP_j = F_j(NE T_j) = \frac{1}{1 + e^{-(net / \theta_j)}} \quad (2.21)$$

$$OP_k = F_k(NE T_k) = \frac{1}{1 + e^{-(net / \theta_k)}} \quad (2.22)$$

กำหนด

$$\theta_j = 0.05 \text{ และ } \theta_k = 0.05$$

θ_j และ θ_k คือ ค่าที่กำหนดความชันของ Sigmoid curve

OP_j คือ ค่า Output ที่ Neuron j ของ Hidden Layer (เป็น Input ของ Output Layer)

OP_k คือ ค่า Output ที่ Neuron k ของ Hidden Layer (เป็น Input ของ Output Layer)

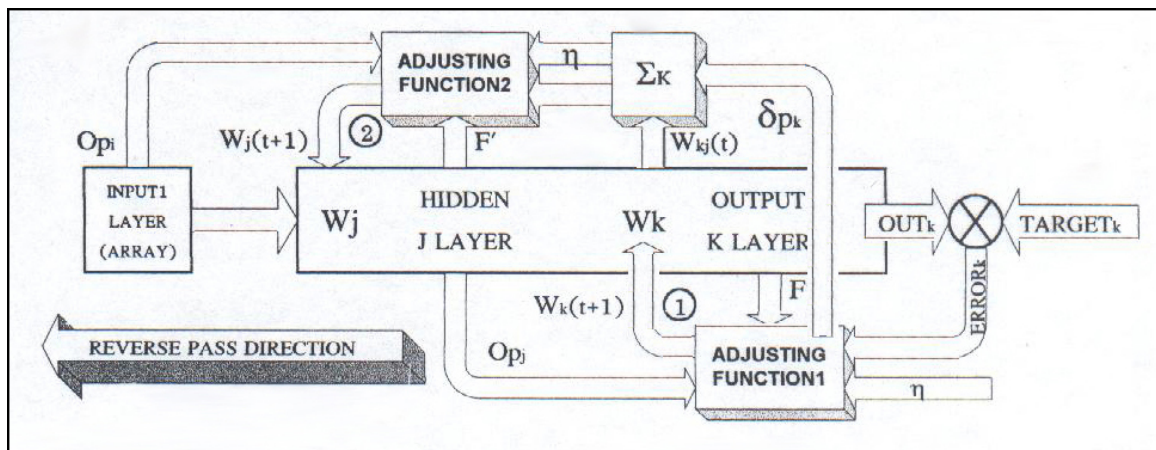
การคำนวณหาค่า Output จะเริ่มที่ Hidden layer ที่ชั้น j ก่อน เมื่อได้ค่า output ของ Hidden layer ครบทุกตัวแล้ว จึงคำนวณหาค่า Output ที่ Output layer โดย Input ของ Output layer ก็คือ Output ของ Hidden layer นั้นเอง โดยใช้การคำนวณทำนองเดียวกันกับ Hidden layer Output ที่ได้จะนำไปเปรียบเทียบกับ Target ที่ต้องการเกิดเป็น Error ขึ้น ค่า Error นี้ จะถูกนำไปใช้ร่วมกับค่าดิฟเฟอเรนเชียล ของ $F(F')$ ใน Output layer เพื่อปรับค่า Weight ให้กับ Output layer ดังจะกล่าวต่อไปในส่วนของ Reverse pass

REVERSE PASS

เป็นส่วนที่ป้อนกลับค่า Error(Target-Output) จากเอาต์พุตเลเยอร์ย้อนกลับมาปรับปรุงค่าน้ำหนักที่เลเยอร์ก่อนหน้านี้ สำหรับนิวรัลเน็ตเวิร์คที่มีมากกว่า 1 เลเยอร์ การปรับปรุงค่าน้ำหนักชั้นถัดลงมา จะใช้ค่าน้ำหนักเดิมของเลเยอร์ที่สูงกว่า ร่วมกับตัวร่วมทางด้าน Output ของเลเยอร์มาคำนวณ (เนื่องจากไม่มีค่า Error มาใช้เหมือน Output layer) ดังจะกล่าวในรายละเอียดดังต่อไปนี้

จากภาพที่ 2.21 การปรับเปลี่ยนค่าน้ำหนักใหม่ จะเป็นไปตามทิศทางของลูกศร และได้ข้อมูลจากค่า Error ที่ Output layer เป็นต้นกำเนิดเพื่อกำหนดทิศทางปรับค่าน้ำหนักให้นิวรัลเน็ตเวิร์ค มีคุณลักษณะสมบัติตามต้องการ ลำดับขั้นตอนของ Reverse pass พอสรุปได้ดังนี้คือ

1. นำ Output layer ไปลบออกจาก Target ได้เป็น Error
2. นำค่า Error มาคูณกับ F' ที่ได้จาก Output layer เรียกว่า δp_k
3. นำค่า δp_k คูณกับ Op_j แล้วคูณกับ η (อัตราการเรียนรู้ 0.01-1)
4. ค่า Weight ค่าใหม่ที่ได้ คือ ค่า Weight เดิมบวกกับผลลัพธ์ ที่ได้จากข้อ 3



รูปที่ 2.21 แสดงไดอะแกรมของการปรับค่า Weight ในชั้น Output layer และ Hidden layer

พอเขียนเป็นสมการใหม่ได้คือ

$$W_{kj}(t+1) = W_{kj}(t) + \eta \delta p_k Op_j \quad (2.23)$$

เพราะว่า $\eta \delta p_k Op_j = \Delta W_{kj} \quad (2.24)$

ฉะนั้น $W_{kj}(t+1) = W_{kj}(t) + \Delta W_{kj} \quad (2.25)$

เมื่อ $W_{kj}(t)$ คือ ค่าน้ำหนักเดิมก่อนปรับ

$W_{kj}(t+1)$ คือ ค่าน้ำหนักหลังปรับ

η คือ อัตราการเรียนรู้

Op_j คือ Output ของ Layer j

δp_k คือ ค่าที่ได้จาก Error คูณกับดิฟเฟอเรนเชียลของฟังก์ชัน F

การปรับค่า Layer ถัดมา (Hidden layer) มีลักษณะคล้ายคลึงกับ Output layer คือ

$$W_{ji}(t+1) = W_{ji}(t) + \eta \delta_{pj} Op_i \quad (2.26)$$

เพราะว่า $\eta \delta_{pj} Op_i = \Delta W_{ji}$ (2.27)

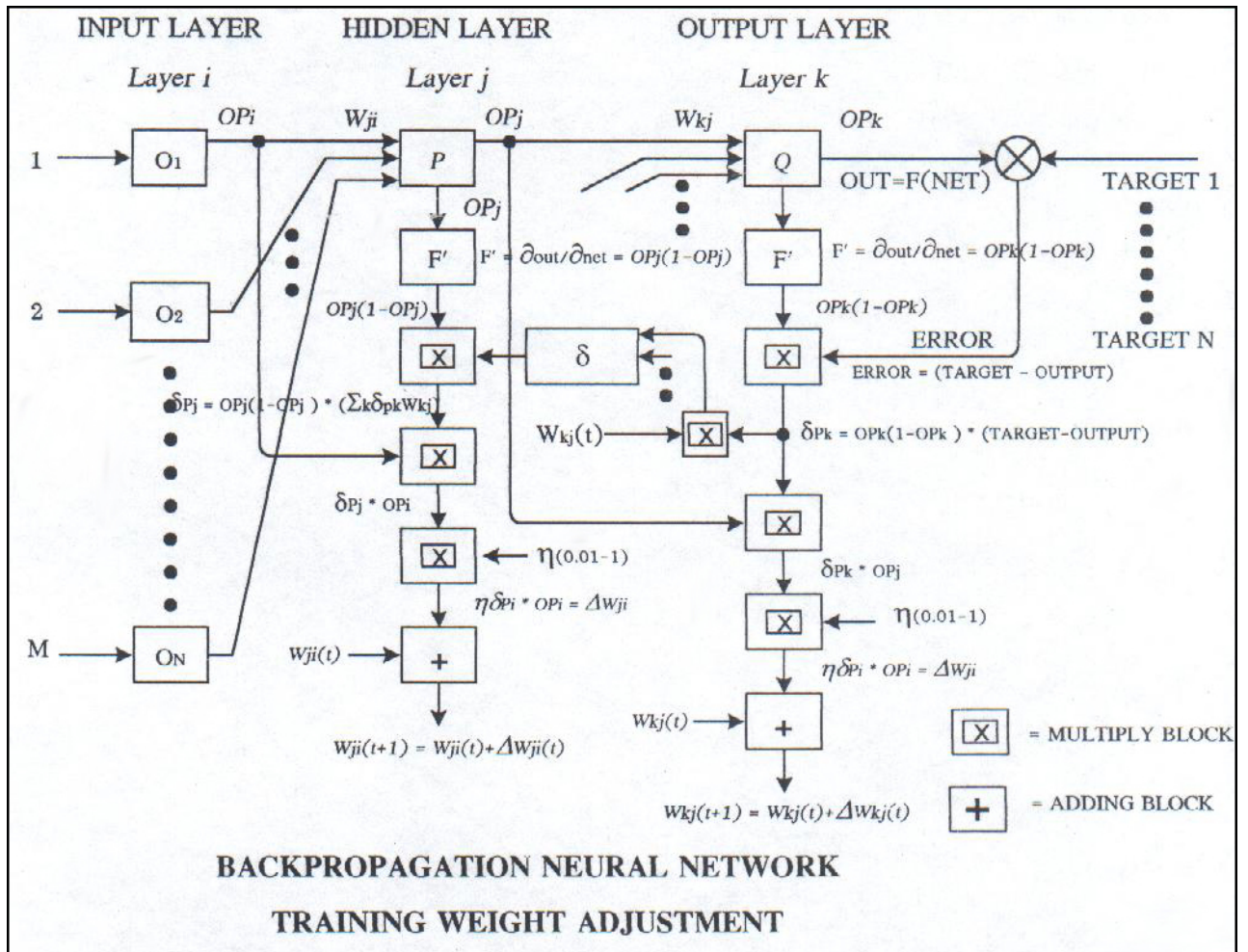
ฉะนั้น $W_{ji}(t+1) = W_{ji}(t) + \Delta W_{ji}$ (2.28)

สมการที่ 2.27 แตกต่างกับสมการ 2.24 ตรงที่ได้มาซึ่ง δ_{pj} เพราะไม่มีค่า Output กับ Target ที่ Hidden layer ค่าดังกล่าวสามารถทดแทนได้ด้วย $\sum_k (\delta_{pk} W_{kj})$

โดย $\delta_{pj} = Op_j(1 - Op_j)(\sum_k (\delta_{pk} W_{kj}))$ (2.29)

เมื่อ $W_{ji}(t)$ คือ ค่าน้ำหนักเดิมก่อนปรับ
 $W_{ji}(t+1)$ คือ ค่าน้ำหนักหลังปรับ
 η คือ อัตราการเรียนรู้
 Op_i คือ ข้อมูลที่ได้จาก Input Layer
 δ_{pj} คือ ค่าควบคุมให้น้ำหนักเปลี่ยนแปลงเข้าสู่จุดที่ดีที่สุด

การปรับค่าน้ำหนักจะปรับทีละเลเยอร์จาก Output layer กลับไปยัง Input layer เมื่อเสร็จสิ้นก็จะกลับสู่ส่วนของ Forward pass อีกครั้ง เพื่อหา Output มาเปรียบเทียบกับ Target และดำเนินการปรับค่าน้ำหนักในส่วนของ Reverse pass เช่นนี้สลับกันไปจนกว่าค่า Error ที่ได้จะลดลงต่ำกว่าค่าที่ต้องการจึงหยุดกระบวนการ ค่าน้ำหนักที่ได้จะเป็นค่าที่เหมาะสมสำหรับทุกคู่ของ Input และ Target ซึ่งพร้อมที่จะนำไปใช้ในกระบวนการตรวจสอบ การปรับค่าน้ำหนักเทรนนิ่งของ Backpropagation Network พออธิบายได้ด้วยไดอะแกรม ดังนี้



รูปที่ 2.22 โปรแกรมแสดงวิธีการหาค่าน้ำหนักที่เหมาะสมของการเทรนนิ่ง
โดยใช้อัลกอริทึมของแบคพรอบพาเกชั่น

2.11.14 เงื่อนไขการหยุดฝึกสอน

การกำหนดเงื่อนไขการหยุดฝึกสอน ทำได้สองกรณี คือ

1. เมื่อผลรวมของค่าผิดพลาดเฉลี่ยระหว่างเอาต์พุตกับเป้าหมายทั้งหมด (SSQERR) ลด
น้อยลง กว่าค่าที่กำหนด โดยกำหนดสมการหาค่าผิดพลาดเฉลี่ยของเอาต์พุตทั้งหมดดังนี้

$$SSQERR = \frac{1}{2} \sum (Target - Output)^2 \quad (2.30)$$

SSQERR คือ ค่า Sum Square Error ที่ได้จาก Output neurons เทียบกับ Target ใช้สำหรับ
บ่งชี้ผลของการฝึกสอนว่าเพียงพอกับการใช้งาน หรือเงื่อนไขที่กำหนดหรือไม่

2. จำนวนรอบของการฝึกสอนดำเนินไปถึงค่าที่ตั้งไว้ ซึ่งจำนวนรอบที่เหมาะสมได้จากการ
ทดลอง (จากที่ศึกษา จำนวนรอบที่เหมาะสมอยู่ในช่วง 2,500 – 5,000 รอบ)